Computers & Security 173 (2027) 105203

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

Full length article

Exploiting CPython's just-in-time optimization: StencilLeak attack

Seehwan Yoo [a], Youngpil Kim [b,*], Chuck Yoo [c]

[a] Dankook University, 152 Jukjeon-ro, Yongin-si, 16890, South Korea
[b] Incheon National University, 119 Academy-ro, Incheon, 22012, South Korea
[c] Korea University, 145 Anam-ro, Seoul, 02841, South Korea

ARTICLE INFO

Keywords:
CPython
Just-in-time compilation
Return-oriented programming
Software security
StencilLeak

ABSTRACT

CPython's experimental copy-and-patch just-in-time (JIT) compiler adds runtime-generated executable buffers to the native code already present in the interpreter, extension modules, and shared libraries. We present, to our knowledge, the first systematic security analysis of CPython's copy-and-patch JIT as a code-reuse surface, evaluated on CPython 3.14 for x86_64. Fixed stencil bytes and patch-site metadata can be cataloged before execution, whereas stencil selection, patched bytes, layout, gadget availability, and addresses require runtime observation. We introduce StencilLeak, a structural, emission-level code-reuse technique in which source structure influences which stencils are emitted while the gadget bytes remain engine- and allocator-controlled. The usable gadget set combines deterministic template-borne gadgets with probabilistic gadgets formed within ASLR-randomized patched pointer immediates and exposed through unaligned decoding. Our threat model assumes that an independent memory-safety flaw or exploit chain provides repeatable JIT-buffer disclosure, writable chain storage, and a controlled transfer with a controlled stack. To isolate the post-hijack stage, an instrumentation harness directly supplies the corresponding memory and control state, locates the required gadgets, constructs a complete native return-oriented programming (ROP) chain from JIT-resident bytes, and executes it from its first gadget. Separately, under the evaluated deterministic workload with garbage collection disabled and a two-hour per-process search budget, a dry-run benchmark measured gadget-set availability: 251 of 500 fresh processes exposed all five required patterns (50.2%). Together, these results establish fixed template bytes and runtime-patched values as complementary components of CPython's copy-and-patch JIT code-reuse surface.

1. Introduction

Python has become one of the most widely used programming languages, praised for its readability, simplicity, and flexibility across application domains. In particular, Python plays a central role in artificial intelligence (AI) research and production, powering frameworks such as TensorFlow (Abadi et al., 2016), PyTorch (Paszke et al., 2019), and scikit-learn (Pedregosa et al., 2011). This ubiquity has made Python the de facto language for machine learning, data science, and cloud infrastructure management.

Despite its strengths, Python's performance has long lagged behind native languages like C++ or Rust. Because Python relies on interpreter-based execution, it exhibits performance bottlenecks, particularly in tight computational loops such as neural network training and inference, prompting extensive research into runtime optimization techniques. Optimizations such as Just-In-Time (JIT) compilation have attracted growing interest.

CPython is a popular implementation of Python, and recent CPython 3.13 and 3.14 include a new JIT optimization technique called "copy-and-patch" as an experimental feature (Bucher and Shannon, 2023; Python Software Foundation, 2024; Shannon et al., 2021). It dynamically translates Python bytecode into native machine instructions at runtime, offering substantial speedups. Specifically, this JIT optimization uses precompiled machine code templates, known as *stencils*, which minimize the runtime compilation overhead. Early benchmarks report performance gains of up to 30% in select workloads (Python Core Developers, 2023) without requiring a full-fledged dynamic compilation infrastructure. Although this JIT optimization is now experimental in CPython, given Python's rapid growth in AI system deployments and the critical need for performance, it warrants a rigorous security evaluation. To our knowledge, no prior work has systematically analyzed the security implications of CPython's copy-and-patch JIT.

Enabling CPython's experimental copy-and-patch JIT adds runtime-generated executable buffers to the native code already present in the

* Corresponding author.
E-mail address: ypkim@inu.ac.kr (Y. Kim).

https://doi.org/10.1016/j.cose.2026.105203
Received 5 April 2026; Received in revised form 21 August 2026; Accepted 7 October 2026
Available online 26 October 2026
0167-4048/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

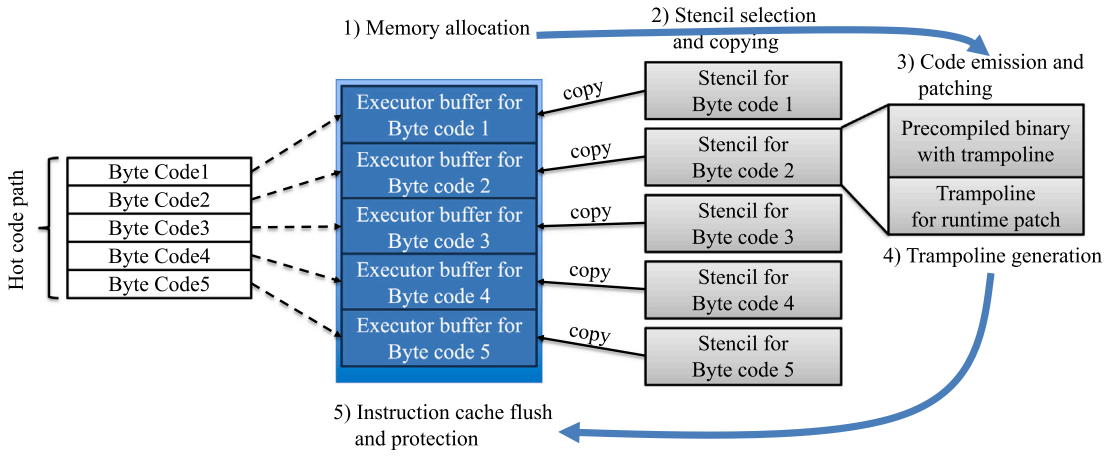


Fig. 1. Overview of CPython's Tier-2 JIT processing.

process. We analyze how this additional region changes the process's code-reuse attack surface. This region is one gadget source alongside the interpreter, extension modules, and shared libraries, not the primary or only one. Our analysis makes the same assumptions as prior code-reuse work: enabling the JIT grants no new privileges and introduces no memory error by itself, so using the characterized surface presupposes an independent memory-safety vulnerability, whose required capabilities we specify in the threat model (Section 4.1). Under these assumptions the paper characterizes the gadgets this JIT-resident surface can supply.

Security risks associated with JIT compilation are well-documented in JavaScript JIT engines (Athanasakis et al., 2015; Maisuradze et al., 2016). CPython is known to be a relatively safe runtime due to its interpreter-based execution. In the context of the new CPython JIT optimization, however, we show that stencil-based code emission adds a partially offline-cataloguable code-reuse surface. We introduce *StencilLeak*, a structural variant of existing code-reuse attacks that treats copy-and-patch emission as a gadget source: the fixed structural template bytes supply always-available gadgets, while the low bytes of ASLR-randomized runtime-patched pointer immediates supply the probabilistic bottleneck and `syscall` gadgets. Given the memory-safety prerequisites assumed in our threat model, we show that a native ROP chain can be assembled and run from these JIT-resident gadgets without injecting any attacker-chosen gadget bytes. Also, we empirically demonstrate that gadget availability is not uniform but probabilistic, depending heavily on the per-process ASLR layout: the input Python source shapes only *how many* runtime-patched pointer immediates a workload emits, not their bytes.

Our Proof-of-Concept (PoC) Python prototype, once the assumed memory-safety prerequisites are supplied, locates the required gadget patterns, assembles the chain, and executes it from the first JIT-resident gadget. Our 500-attempt evaluation locates all required gadget patterns in 50.2% of runs (a gadget-set discovery rate for this configuration, not a payload-execution rate); under an unverified iid assumption, this yields a model-derived 99.2% probability of at least one gadget-complete run within 7 attempts (see Section 5 for detailed results).

Scope of this paper. Our focus is exclusively on the offensive security implications of template determinism. While we systematically demonstrate structural vulnerabilities and outline the theoretical limitations of potential mitigations, proposing and evaluating a comprehensive, low-overhead defense mechanism within the CPython core is left as future work.

This paper makes the following contributions:

- Systematizes the threat model of copy-and-patch JIT architectures, distinguishing *template determinism* from traditional JIT spraying and JIT-ROP.
- Demonstrates, under ASLR and AppArmor and given the assumed memory-safety prerequisites, harness-assisted construction and execution of a native `execve` ROP chain from JIT-resident gadgets, built from fixed structural shim-epilogue bytes together with the low bytes of ASLR-randomized runtime-patched pointer immediates, without injecting attacker-chosen gadget bytes.
- Quantifies stencil predictability on x86_64: easy gadgets always present, bottleneck gadgets (`pop rax/rdx`) appear in 72.4% /65.0% of attempts.
- Validates StencilLeak at scale: 500-attempt evaluation locates all required gadget patterns in 50.2% of runs (a gadget-set discovery rate), a model-derived 99.2% within 7 attempts under an iid assumption.
- Provides a critical analysis of why standard mitigations (e.g., nop-insertion, re-randomization) incur prohibitive costs in Python environments.

The remainder of this paper is structured as follows: Section 2 provides the necessary background and reviews related work. Section 3 details the architecture of CPython's copy-and-patch JIT and its core functionalities. Section 4 presents our StencilLeak exploitation methodology and threat model. The experimental methodology underpinning our validation campaign is documented in Section 5. In Section 6, we discuss feasible mitigation strategies. Finally, Section 7 concludes the paper.

2. Background and related work

In this section, we provide the necessary background on JIT optimization and review related work to highlight the unique contributions of our study. Unlike traditional JIT attacks that focus on dynamic code generation or JIT spraying with immediates, our work addresses a new threat of predictable stencil reuse in copy-and-patch JITs.

2.1. Overview of JIT optimization

Python 3.13 introduced a new feature called copy-and-patch Just-In-Time (JIT) compilation. This represents a significant advancement in optimizing Python's performance. The JIT compiler uses precompiled machine code templates, which are called "stencils". These stencils contain empty slots, or placeholders, known as "holes". The compiler patches these holes dynamically while the program is running. The patches insert specific values such as literals, operands, and jump addresses. These values are customized for the specific execution paths of the Python bytecode (Shannon et al., 2021).

CPython executes code using a two-tier model:

- **Tier-1 (Interpreter)**: The traditional CPython interpreter profiles to identify frequently executed ("hot") functions while executing Python bytecode.
- **Tier-2 (JIT)**: Once functions are marked as hot, they are promoted to JIT compilation. Their bytecode is first translated into an optimized intermediate representation (IR), comprising micro-operations ("uops"). These uops are then lowered into native machine code using the copy-and-patch mechanism.

CPython does not generate native code from scratch. Instead, it uses a stencil-based approach for code generation. Stencils are pre-compiled machine code templates derived directly from the interpreter's micro-operation (uop) definitions. This strategy balances performance optimization with maintainability, significantly reducing the complexity and compilation overhead from real-time code generation.

The complete Tier-2 JIT process consists of the following steps in Fig. 1:

1. **Executor Buffer Allocation**: At runtime, the JIT compiler allocates a dedicated memory region (the executor buffer) for code emission and patching. This region contains native code, trampoline stubs, and associated data.
2. **Stencil Selection and Copying**: The system maps each micro-operation (uop) to a corresponding stencil template. For every uop in the optimized trace, the JIT compiler selects the matching stencil. It then copies that stencil into the allocated buffer. CPython includes 274 stencils for common operations on the x86_64 architecture. Because the system reuses these templates, the resulting instruction sequences are deterministic, remaining identical across different compilations.
3. **Code Emission and Patching**: Each stencil's `_emit()` function copies the template into the allocated memory and dynamically patches placeholders with actual values, such as relative jump offsets, constants, and data references.
4. **Trampoline Generation**: If external symbols or runtime helper functions are required, the JIT uses trampolines. Stencils use trampolines for indirect jumps (e.g., `jmp rax`) to absolute 64-bit addresses.
5. **Instruction Cache Flush and Protection**: Once native code is emitted, memory regions are marked executable, and the instruction cache is flushed to ensure CPU coherency. The generated machine code becomes active and is invoked through function pointers at run-time.

This structured approach, through the reuse of stencil templates, provides substantial runtime efficiency across various computational workloads. Nonetheless, enabling CPython's experimental copy-and-patch JIT expands CPython's existing code-reuse attack surface by adding JIT-generated executable buffers.

2.2. Related work

In this section, we review the landscape of JIT security research to position our work. We distinguish our work from prior studies by examining three fundamental dimensions: the underlying JIT compilation mechanisms, the evolution of attack techniques, and the effectiveness of existing defense strategies against copy-and-patch architectures.

2.2.1. Differences in JIT mechanisms

Dynamic language JITs such as V8 (JavaScript) and HotSpot (Java) generate machine code through multi-stage compilation pipelines that include parsing, intermediate representation (IR) construction, optimization passes, and register allocation. These stages rely on runtime profiling feedback, speculative optimizations, and dynamic deoptimization. Consequently, the generated code varies significantly across executions, even for identical source code.

Existing Python JITs follow similar dynamic generation paradigms. **Tracing JITs** like PyPy (Bolz and Rigo, 2009) record frequently executed paths at runtime and emit specialized machine code based on observed type information, producing different traces for different execution patterns. **LLVM-backed compilers** like Pyston (Pyston Team, 2021) and Numba (Numba Developers, 2024) delegate code generation to LLVM's optimization infrastructure, where the output varies with optimization levels, inlining decisions, and target-specific code generation. **Domain-specific JITs** like TorchDynamo (PyTorch Team, 2023) and MesaPy (MesaPy Contributors, 2019) generate code tailored to specific workloads such as machine learning graph execution or memory-safe execution.

In contrast, CPython's copy-and-patch JIT adopts a fundamentally different approach. Originally proposed by Xu and Kjolstad (2021), the copy-and-patch technique is a fast compilation method for high-level languages. It copies pre-compiled binary blocks (stencils) directly into memory and patches only the necessary operands at runtime. Unlike the JITs described above, the emitted code sequences are deterministic: the same stencil always yields an identical byte sequence. This determinism underpins our attack: while dynamic JITs produce unpredictable code that resists offline analysis, copy-and-patch JIT produces predictable patterns that can be cataloged and exploited. Consequently, StencilLeak is specific to copy-and-patch architectures; it does not apply to dynamic JITs such as PyPy, Pyston, or Numba, where code generation varies across executions.

Note that not every gadget is produced deterministically. Some arise instead from the *object addresses* that runtime patching writes into the emitted code, so they are generated randomly according to each process's ASLR layout. The bottleneck gadgets `pop rax`/`pop rdx` are of this kind, *probabilistic* rather than deterministic (Section 5.5).

2.2.2. Differences in attack techniques

Traditional JIT attacks do not directly translate to copy-and-patch architectures. Table 1 summarizes the fundamental differences between prior JIT exploitation techniques and our StencilLeak vector. All three vectors in Table 1 additionally assume the independent disclosure and control-flow-transfer primitives of the threat model (Section 4.1); the table contrasts only the gadget source and what the attacker supplies. StencilLeak therefore assumes the same prerequisites as JIT-ROP and adds source-driven emission shaping with an offline stencil catalog.

The emergence of template determinism as a vulnerability extends beyond CPython. Recent trends in highly constrained runtime environments, such as eBPF (Extended Berkeley Packet Filter) and WebAssembly (WASM), have also seen shifts toward predictable JIT compilation strategies to guarantee safety and low latency (Lehmann et al., 2020; He et al., 2023). For instance, eBPF JIT engines similarly rely on static instruction mapping, which researchers have recently begun to scrutinize for code-reuse opportunities (He et al., 2023). Our systematization of StencilLeak highlights a timely, overarching challenge in modern system design: as JITs move toward deterministic, template-based backends for predictable performance, they inadvertently resurrect static ROP surfaces inside the runtime.

Foundational JIT attacks like JIT spraying (Blazakis, 2010; Lian et al., 2015) largely rely on the attacker's ability to inject specific immediate values (constants) that, when executed as code, form malicious gadgets. This approach effectively treats data constants as a delivery vehicle for malicious instruction sequences, as comprehensively surveyed by Gawlik and Holz (2018).

JIT-ROP (Snow et al., 2013), by contrast, injects nothing: using a memory-disclosure primitive, the attacker's script scans the victim's existing executable code pages page by page at run time and assembles a ROP chain just-in-time from the gadgets it finds, which may lie within existing instruction bytes as well as at instruction boundaries.

StencilLeak introduces a distinct attack vector, a structural, emission-level variant of code reuse. Unlike traditional JIT spraying, the JIT-resident gadgets StencilLeak harvests are built not from attacker-chosen

Table 1
Comparison of JIT exploitation vectors.

Attack Class	Gadget bytes come from	What the attacker controls	Runtime code disclosure
JIT Spraying (Blazakis, 2010)	Attacker-chosen constants, mis-decoded unaligned	The injected constant values	Not required for the gadget bytes (attacker-chosen); a target address is still needed
JIT-ROP (Snow et al., 2013)	Existing victim code, gadget semantics unknown in advance	A memory-disclosure primitive and a harvesting script	Required: recursive, page-by-page analysis of unknown code
StencilLeak (ours)	Engine/allocator bytes: structural opcodes, ASLR-randomized pointer immediates	The submitted Python source, shaping which stencils are emitted (not the gadget bytes)	Required: byte-level matching of known patterns in disclosed buffers; candidate origins known offline

constant *values* but from the engine's structural opcodes and allocator-controlled pointer immediates. A StencilLeak attacker controls *code patterns*, specifically the structure of the input Python source code. This input structure influences which stencils the JIT compiler selects and how it lays them out in memory. While StencilLeak leverages Return-Oriented Programming (ROP) principles as its payload delivery mechanism, its core contribution is to characterize this copy-and-patch JIT as a structural, emission-level code-reuse surface in which source structure shapes which stencils are emitted while the reused gadget bytes remain engine- and allocator-controlled.

This fundamental difference is pivotal to the feasibility of our attack. Although each stencil is deterministic and independent by itself, the *combination and layout* of stencils vary with the input Python code. Critical gadgets such as `pop rax` and `pop rdx` do not exist within any single stencil. Instead, they are the low bytes of runtime-patched `mov-abs` pointer immediates (engine- and allocator-controlled) surfaced by unaligned decoding, not the byte boundaries of stencil sequences. The attacker's goal shifts from "what data to inject" to "which stencil combinations to induce". Because these gadget bytes are engine- and allocator-controlled rather than attacker-supplied, they fall outside the reach of anti-spraying heuristics aimed at injected constants.

2.2.3. Differences in defense techniques

Prior security research on Python JITs has focused primarily on runtime hardening (e.g., memory safety in MesaPy (MesaPy Contributors, 2019)) or execution isolation (e.g., SGX enclave sandboxing in ScriptShield (Wang et al., 2019)), rather than analyzing systematic code-reuse exploitation techniques.

Standard hardening techniques such as constant blinding (Jauernig et al., 2014) or instruction alignment enforcement (Yee et al., 2009) are designed to mitigate JIT spraying by neutralizing data-embedded gadgets. Similarly, fine-grained permission management schemes like NoJITsu (Park et al., 2020) and safe code generation frameworks like JiuJITsu (Zhang et al., 2024) reduce the attack surface by limiting writable windows or constraining register choices. However, these defenses are ineffective against StencilLeak as they target attacker-injected patterns. In contrast, our gadgets are inherent to the legitimate JIT code structure, which is validly emitted and marked executable.

Hardware-based mitigations such as ARM Pointer Authentication (PAC) (Liljestrand et al., 2019), SGX-based hardening (Frassetto et al., 2017), and software CFI (Abadi et al., 2005; Niu and Tan, 2014) could theoretically stop ROP chains. Forward-edge policies do constrain indirect `call/jmp` targets, and a backward-edge shadow stack constrains `ret` targets independently of a gadget's origin; we analyze which transfers each mechanism covers in Section 6.3. However, forward-edge mechanisms alone struggle to distinguish between a legitimate function call and a malicious jump to a valid gadget within a stencil, since both target addresses reside in legitimately executable memory regions.

Finally, existing JIT-focused security testing tools such as LangFuzz (Holler et al., 2012), Fuzzilli (Groß et al., 2023), and FuzzJIT (Wang et al., 2023) are designed to discover memory corruption vulnerabilities through crash detection. These tools do not catalog the inherent gadget availability within valid code emissions, a gap that StencilLeak addresses by systematically analyzing stencil byte sequences for exploitable instruction patterns.

3. Architecture of CPython's copy-and-patch JIT

3.1. Architecture and mechanisms

CPython's copy-and-patch JIT compiler explicitly manages executable memory regions through careful allocation and protection handling according to its stencil-based emission strategy as follows. We provide only the details necessary to understand our exploits.

3.1.1. Executor buffer allocation

Native code stencils are stored as precompiled instruction sequences (`code_body[]`) along with associated metadata (`data_body[]`). During JIT compilation, these are copied into dynamically allocated memory. On macOS, `mmap()` with `MAP_JIT` (Apple Inc., 2024) provides hardware-enforced separation between writing and execution; Linux lacks `MAP_JIT`, so pages are mapped RW, patched, then transitioned to RX with `mprotect()`:

```
unsigned char *memory = mmap(NULL, size,
  PROT_READ | PROT_WRITE,
  MAP_ANONYMOUS | MAP_PRIVATE /* + MAP_JIT on macOS */,
  -1, 0);
/* ... emit & patch ... */
mprotect(memory, size, PROT_READ | PROT_EXEC);
```

3.1.2. Stencil selection and copying

Next, JIT selects the appropriate stencil for the bytecode in the hot code path. For instance, when an integer addition appears in the hot code path, the JIT selects `LOAD_CONST()` and `BINARY_OP_ADD_INT()` stencils. CPython provides 274 stencils for commonly used operations on x86_64 architecture.

3.1.3. Code emission and patching

Once the stencil is selected, stencil code and data blocks are copied to the executor buffer. This copy operation is code emission. Each stencil has its own emission function. For example, `emit__BINARY_OP_ADD_INT()` is the function that emits the template code for an integer addition operation (`BINARY_OP_ADD_INT`). A code emission for the x86_64 architecture is below:

The emitted stencil code begins with a standard function prolog and increases the profiler counter statistics:

```
pushq %rbp # store stack frame
movabsq $0x0, %rax # load _Py_stats
testq %rax, %rax # NULL check
je 0x21 # skip if NULL
incq 0x9c958(%rax) # update stats counter
incq 0x1ce628(%rax) # update stats counter
```

Explicit type checks ensure operands are of the `PyLong` type, involving bitmasking pointers and comparisons:

```
movq -0x10(%r13), %rbx # load first operand
movq %rbx, %rdi # copy operand to rdi
andq $-0x2, %rdi # get PyObject ptr
movabsq $PyLong_Type, %rcx
                # load PyLong_Type to rcx
cmpq %rcx, 0x8(%rdi)
                # compare ob_type is PyLong
jne 0x179
                # if not PyLong_Type, jump out
```

Similarly, the system loads the second operand into the `rsi` register and verifies that it is a PyLong. Finally, `_PyLong_Add` function is invoked as follows:

```
callq *(%rip) # indirect jump _PyLong_Add
mov 0x40(%r12), %r13 # result on value stack
test %r13,%r13 # check result
je 0x1f4 # jump out if error
```

In this process, JITed code should be patched to reference symbols like `_Py_stats`, `PyLong_Type`, `_PyLong_Add`. The original stencil template is initialized with zeros. At every JIT code emission, the `patch_64()` function overwrites the correct value on the stencil template, enabling real jumps and reference objects at runtime.

3.1.4. Runtime patching mechanism

The JIT compiler employs a runtime patching mechanism to resolve address references within the emitted stencils. Patch addresses are computed relative to the stencil's data segment (`data_body[]`), which serves as a JIT-managed function pointer table (similar to a Global Offset Table or GOT) for runtime symbols such as `_Py_stats`, `PyLong_Type`, `_PyLong_Add`, and error handlers. Three patch functions handle different relocation types: `patch_64` writes 64-bit absolute addresses, `patch_32r` handles relative branches, and `patch_x86_64_32rx` sets up RIP-relative indirect calls through data slots:

```
// Patches to data segment (function pointer slots):
patch_64(data + 0x320, (uintptr_t)&_PyLong_Add);
    // Function pointer
patch_64(data + 0x328, (uintptr_t)&__assert_fail);
    // Function pointer

// Code segment patches:
patch_64(code + 0x3, (uintptr_t)&_Py_stats);
    // Direct absolute address in code
patch_64(code + 0x2e, (uintptr_t)&PyLong_Type);
    // Direct absolute address in code

// RIP-relative indirect call through data segment (GOT-
     like pattern):
patch_x86_64_32rx(code + 0x74, (uintptr_t)data + 0x31c);
    // Points to data slot
// That data slot at 0x320 contains &_PyLong_Add
```

With ASLR, functions can be relocated at runtime. An indirect call to a relocated function involves two steps: first, the function pointer is stored in the `data_body[]` slot (e.g., `data + 0x320` above), then the code segment issues an indirect call through that slot to invoke the target function.

3.1.5. Instruction cache flush and protection

After stencil emission and dynamic patching, the allocated memory explicitly transitions to a secure executable state (RX), disallowing further modifications to mitigate security risks. This transition involves the `mprotect()` system call or equivalent, coupled with an instruction cache flush to maintain CPU coherency:

```
__builtin___clear_cache((char *)memory,
                        (char *)memory + size);
mprotect(memory, size, PROT_READ | PROT_EXEC);
```

3.2. Attack surface analysis

The introduction of the copy-and-patch JIT optimization in CPython significantly alters its threat model by generating executable memory regions during runtime. The JIT regions introduce specific vulnerabilities due to their characteristics.

Repeated emission of stencil templates in `code_body[]` is the primary source of reusable ROP gadgets, while patchable slots add control-flow redirection points. RW→RX remains a potential W^X weakness if `mprotect()` can revert protections.

In this section, we analyze the expanded attack surfaces of CPython's Copy-and-Patch JIT optimization below, including the practical limitations of current protections.

Predictable emission of native code templates. The stencil-based JIT approach repeatedly reuses precompiled native code templates stored initially in `code_body[]`. This reuse ensures that instruction sequences are **deterministic**. That is, the binary content of each stencil is fixed at build time. This determinism renders the code emission **predictable** to an attacker, effectively transforming the JIT engine into a static library of exploitable gadgets, unlike dynamic JITs where code generation varies.

Runtime-patchable immediates. Dynamically patched constants and immediate values can be encoded as part of an instruction or an indirect jump target. Patched values may carry sensitive information or be attacker-controlled under certain conditions. This patching process, although integral to the stencil JIT approach, provides potential entry points for injecting attacker-supplied data.

Separated copy vs. Patch regions. Copy occurs in the stencil code segment (`code_body[]`), while patching targets data slots (`data_body[]`) that hold addresses and immediates. The clear boundary between executable code and patchable data aids defenders conceptually, but it also exposes two distinct attack surfaces: predictable reusable code for gadget harvesting, and writable data slots whose contents can redirect control flow if corrupted.

Indirect branching via indirect jumps. JIT-emitted indirect branch instructions (e.g., `jmp rax` on x86_64) can be abused if an attacker controls the relevant registers or memory locations to redirect control flow. Under the independently assumed disclosure and launch capabilities, such a branch could participate in a code-reuse chain that reuses existing executable mappings without making those mappings writable.

Availability of ROP gadgets. Repeated stencil emission can produce short instruction sequences or "gadgets". After a disclosure primitive resolves their runtime addresses, these sequences can participate in code-reuse chains. ASLR affects these gadgets differently by type: the fixed structural gadgets (`pop rsi`/`pop rdi`) are stencil-template bytes present deterministically in every executor, with only their absolute address randomized by ASLR, whereas the bottleneck gadgets (`pop rax`/`pop rdx`) are the low bytes of ASLR-randomized runtime pointers, so their very presence depends on the per-process ASLR layout and is probabilistic (Section 5.5). In either case the disclosure supplies the addresses and patched-byte observations that ASLR otherwise withholds, while a chain assembled from the JIT mappings reuses executable bytes without changing those mappings to writable. This statement concerns the chain itself. The current PoC uses a separate synthetic launcher in an RWX mapping, so the complete demonstration does not establish operation under strict W^X; this is a limitation of the evaluation harness.

Reversible memory protections. CPython switches JIT pages from RW to RX after patching to enforce W^X protection. However, this protection is often reversible. For instance, any process that can call `mprotect()` can flip pages back to RW to inject malicious code (One, 1996; Blazakis, 2010; Chen et al., 2011). Even strictly enforced mechanisms like Apple's `MAP_JIT` cannot completely eliminate this risk: to support proper JIT functionality, they must fundamentally allow permission toggling, a capability that attackers can abuse.

Concurrent writable-window (TOCTOU) hypothesis. A free-threaded implementation may warrant a separate TOCTOU analysis of the interval in which a JIT buffer is writable before it becomes executable. Such an attack requires a concurrent thread to obtain a write path to that buffer, a capability outside the evaluated CPython configuration. If present, the result would be code injection rather than StencilLeak's reuse of RX bytes. The ordinary RW-then-RX transition remains consistent with W^X because the mapping is not writable and executable at the same time.

Summary of threats. The CPython JIT optimization reduces the entropy relied upon by modern memory protections, effectively bifurcating the attack surface into two categories:

1. **Code Injection (via Reversibility):** Attackers may exploit reversible memory protections (e.g., via `mprotect`) to inject malicious code. As discussed, strict policy-level controls (e.g., Seccomp, AppArmor) or hardware-enforced exclusionary execution (e.g., `MAP_JIT`) can raise the bar for this vector, though they cannot eliminate the toggling primitives required by the JIT.
2. **Code Reuse (via Predictability):** The deterministic nature of stencils transforms the JIT region into a static library of gadgets (StencilLeak). Unlike code injection, this vector (StencilLeak vector) relies solely on existing, legitimately executable code.

Crucially, even if a deployment successfully hardens against code injection, the StencilLeak vector persists because it reuses the engine's own executable bytes: the structural easy gadgets are template code that *must* exist for the JIT to function, while the bottleneck and `syscall` gadgets, though only probabilistically present, likewise come from engine- and allocator-controlled bytes rather than injected code. Therefore, this paper focuses on characterizing and exploiting this more resilient threat: the availability of reusable gadgets in the emitted JIT code.

4. StencilLeak attack

In this section, we detail StencilLeak attack that systematically exploits CPython's stencil-based JIT optimization.

StencilLeak Gadget: A JIT-resident ROP gadget arising from copy-and-patch emission, in two origin classes: (a) *deterministic structural* gadgets from fixed template bytes copied verbatim, with stable relative offsets (`pop rsi/pop rdi`); and (b) *probabilistic, patch-borne* gadgets from the low bytes of ASLR-randomized runtime-patched pointer immediates, present only for particular per-process ASLR layouts (`pop rax/pop rdx`, and `syscall`). Only class (a) is predictable from deterministic stencil reuse; class (b) varies across compilations and processes (Section 5.5).

StencilLeak Attack: A code-reuse technique that chains StencilLeak Gadgets discovered in JIT-resident executable (RX) buffers into a native ROP chain. Because the chain reuses already-executable bytes, it is consistent with W^X by construction; it presupposes the disclosure, writable-storage, and control-transfer capabilities of the threat model (Section 4.1), and the defenses relevant to it are forward-edge CFI and a backward-edge shadow stack rather than W^X (Section 6.3).

With these concepts defined, we outline our threat model and assumptions, followed by detailed examinations of attack vectors. We then present combined attack scenarios that illustrate how these vectors can be orchestrated to compromise CPython's security.

4.1. Threat model

Scope of the analysis. Interpreter-only CPython already executes native code in the interpreter, extension modules, and shared libraries. Enabling the experimental copy-and-patch JIT adds executor buffers containing runtime-generated machine code. We isolate and characterize this newly added executable region, including how it changes the code-reuse surface alongside the pre-existing native regions. Its security consequence in a particular deployment is determined by the combination of the properties characterized here, the permissions and restrictions of the affected process, and an independent memory-safety vulnerability.

Deployment example and source influence. A deployed Python UDF service illustrates one interface pattern in which users submit Python functions that execute under service-imposed restrictions. Snowflake is a cloud-hosted data platform that provides data storage, processing, and a SQL query engine. It lets users extend SQL query processing with user-defined functions (UDFs); a Python UDF implements the function logic as a Python handler that can be invoked from a SQL statement. The Python UDF documentation describes inline user-supplied Python handlers, and the security guidance states that handler code executes in a restricted engine. Because in-process, language-level Python confinement is not by itself a robust boundary (Stinner, 2013), such services restrict the exposed interface and rely on the process's operating-system policy for isolation; the security relevance of a native code-reuse chain is precisely that it can reach an operation the interface withholds but the operating-system policy permits. A hosted interactive notebook service such as Jupyter Lab is a second instance of the same pattern, with users running their own Python under platform-imposed restrictions. These deployment patterns motivate the source-influence condition considered in our analysis; we present them as illustrative interface patterns only, not as evaluated attack scenarios. We separately consider a JIT-enabled CPython process in which a user can submit and repeatedly invoke a function; this affects which functions become hot and which stencils may be emitted. The disclosure, writable storage, and native control-flow capabilities required to use the resulting JIT surface are a separate requirement from the source-level influence above.

Required memory-safety capabilities and runtime information. Our threat model assumes the attacker holds an independent memory-safety vulnerability that provides both a memory-corruption primitive and a memory-disclosure primitive, whether from a single bug (such as a use-after-free) or an exploit chain of several; the analysis assumes this vulnerability rather than discovering or exploiting it. Together these must enable (i) repeatable disclosure sufficient to locate and inspect the relevant JIT executor buffers, (ii) known writable storage for the chain and its data, and (iii) control of a saved return address and the following stack words, or an equivalent control transfer together with a controlled stack. For that transfer to reach and run the chain we further assume, as an execution prerequisite, that userspace shadow stack is disabled and that no forward-edge CFI policy rejects the selected control transfer (Section 6.3). The disclosure supplies the concrete runtime addresses needed under ASLR and reveals the patched bytes present in the emitted buffers; fixed stencil information alone supplies neither. We do not assume a privileged native-access facility such as `ctypes` or attacker-supplied compiled extensions using the CPython API and ABI for this purpose. This is the same class of capability assumed by foundational JIT-ROP (Snow et al., 2013). That work enumerates the same prerequisites (a scripting environment running attacker-provided code, a repeatedly usable memory-disclosure primitive, and a control-flow-hijacking primitive, i.e. a "vulnerable entry point") and, as we do, assumes the initial control transfer as an out-of-scope prerequisite while contributing only the subsequent code-reuse stage. In the Discussion ("Prerequisites for Reliable Exploitation"), we demonstrate without weaponization that a public, already-patched CPython use-after-free provides the assumed disclosure, write, and control-transfer capabilities, but not the controlled stack additionally required for a full launch.

Role of the stencils. CPython's copy-and-patch JIT uses precompiled stencils whose fixed opcode bytes and runtime patch locations can

be analyzed before target execution. Runtime disclosure then reveals which stencils were selected, together with the final patched bytes, emitted layout, gadget availability, and absolute addresses. Combining these two sources of information allows StencilLeak to characterize how this additional executable region supplies reusable machine-code sequences at runtime.

Realizability and measured result. In the evaluation, an instrumentation harness supplies the assumed disclosure, writable-storage, and control-transfer effects directly, so that the added JIT surface can be measured without first exploiting a real bug. This harness, detailed in Experiment 5 (Section 5.6), uses convenience facilities such as `_opcode` and `ctypes` purely as evaluation scaffolding; consistent with the threat model above, these are not capabilities the attacker is assumed to hold. The attack does not depend on them: the same disclosure, write, and control transfer are what an independent memory-corruption vulnerability provides, and a public, already-patched CPython use-after-free (CVE-2026-6100, characterized in the Discussion) realizes them without `_opcode` or `ctypes`, though we do not claim that any specific deployment contains such a bug. Deployed Python UDF platforms and hosted notebooks such as Jupyter Lab let users submit and repeatedly invoke Python, providing the source influence assumed here. The measured result is that, once the assumed memory and control state is supplied, the added JIT surface supports runtime gadget identification, complete chain construction, and native chain execution.

4.2. Attack vectors

The following attack vectors enable a StencilLeak attack:

Predictable Gadget Emission: The copy-and-patch mechanism inherently produces deterministic machine codes: each stencil maps to a fixed, pre-compiled byte sequence. Consequently, if a specific stencil is emitted, the offsets of any gadgets within it can become predictable. However, the *presence* of a given stencil in the JIT region is not guaranteed; it is contingent on the input Python code triggering that specific stencil. Thus, a key distinction emerges: the *content* of a stencil is deterministic, but its *presence* is probabilistic.

Patch-Borne Gadget Formation: A second, dynamic source of gadgets is runtime patching itself. When the JIT resolves an operand, it writes the object's address into a `movabs` immediate; because that address is randomized by the process's ASLR, any gadget byte pattern that forms within it is not produced by a stencil deterministically, but only under particular per-process layouts. The input workload can raise the *number* of such patched immediates, and hence the per-attempt *probability* that one forms a required pattern, but never makes a specific one certain.

JIT Memory Disclosure: CPython exposes JIT region metadata through internal APIs such as `_opcode.get_executor()`, revealing executor base addresses and sizes. This disclosure enables attackers to compute absolute gadget addresses from known relative offsets.

Unaligned Gadget Discovery: On x86_64, variable-length instruction encoding allows byte-shifted decoding to surface gadget-like patterns at non-instruction boundaries. Applied to the fixed template bytes and to the patched immediates above alike, it exposes sequences such as `0xc3` (ret) or `0xff` (indirect jump prefix) that aligned decoding misses, expanding the exploitable gadget space beyond aligned instructions.

ASLR Circumvention: ASLR randomizes the runtime *addresses* of the JIT region, so any candidate gadget, in aligned code or within a patched slot, must be located against the JIT base recovered through the disclosure above; once that base is known, ASLR no longer conceals a gadget that is present. This address-hiding role is distinct from, and does not contradict, the role ASLR plays in Patch-Borne Gadget Formation above, where it instead randomizes *which* patch-borne gadgets form at all. Because patched slots hold live pointer values rather than fixed template bytes, the patched data regions must be scanned as thoroughly as the code.

4.3. Exploitation workflow

To facilitate the StencilLeak attack, we use an orchestration script to automate the workflow across three stages (Fig. 2):

1. Target and Orchestrator (Left Panel): The *Orchestrator* is a Python controller that manages the overall attack campaign. It defines the *Target gadgets* (e.g., `pop rax`, `pop rdi`) required for the ROP chain. It spawns N parallel *exploit instances* to distribute the workload and aggregates success statistics.

2. Exploit Logic (Center Panel): Within each isolated instance, the *Exploit Logic* component drives the attack. It employs a custom `JITFunctionGenerator` paired with a `StencilOptimizer` to iteratively generate randomized Python functions containing high-density gadget patterns (e.g., recursive calls, dictionary manipulations). The logic then inspects the internal `code_body[]` of the resulting JIT objects. When a target gadget is discovered, it is stored in the *found_gadgets* dictionary (e.g., mapping `'pop rax'` to its resolved address). This dictionary serves as the local state for each instance, tracking which parts of the chain are currently available.

3. Output and Execution (Right Panel): This stage handles both result logging and final exploit execution. It records execution metrics and discovered gadgets into detailed *log files* for post-mortem analysis. Upon completing the gadget set, the instance synthesizes the *ROP Stack Layout*, a crafted stack frame containing the resolved gadget addresses and the syscall number (59), and triggers the chain. The attack results in the successful spawning of a shell via invoking `execve(''/bin/sh'')`.

5. Experimental evaluation

5.1. Experimental setup and instrumentation

All experiments run on the x86_64 platform in Table 3, with ASLR and AppArmor enforced. Our toolchain warms Tier-2 JIT, dumps executor buffers, and scans them with Capstone to harvest gadgets. Our Python-based exploit prototype uses only standard Python modules, keeps JIT pages RX, and relies on a minimal ABI-bridge stub; no external library for additional gadgets are used.

We design six experiments that progress from static analysis to practical exploitation: (1) static decode of all 274 stencil templates, (2) dynamic patching effects on gadget counts, (3) byte-shifted decoding at offsets 0–7, (4) patch-impact analysis tracing the origin of the bottleneck gadgets, (5) harness-assisted construction and execution of a JIT-resident ROP chain, and (6) large-scale evaluation of 500 independent attack attempts. For the runtime experiments, each run regenerates functions from a fixed high-yield template and analyses the emitted executor buffers, which vary per run under ASLR to ensure statistical independence (per-experiment function and run counts appear in each experiment's setup; for example, Experiment 2 uses 100 functions across three runs and the large-scale campaign of Experiment 6 regenerates 3000 functions per attempt). Experiment 4 instead uses a smaller set of branch-dense functions tailored to origin tracing (Section 5.5).

5.2. Experiment 1: Static stencil template analysis

Before examining the runtime behavior, we statically decode every `code_body[]` entry in `jit_stencils.h` using Capstone to quantify what the templates alone provide. This pass builds a baseline catalog of mnemonic frequencies and highlights what is and is not available without any runtime patching.

Table 2 summarizes the key findings: only 4 of the 274 templates end with a `ret` (tail), while 90 templates contain `ret` somewhere in their code; none include `pop rbx; ret` or `pop rcx; ret` in aligned static analysis; only one stencil template (`shim`) exposes `pop rdi; ret` pattern; and none emit indispensable gadgets such as `pop rax;`

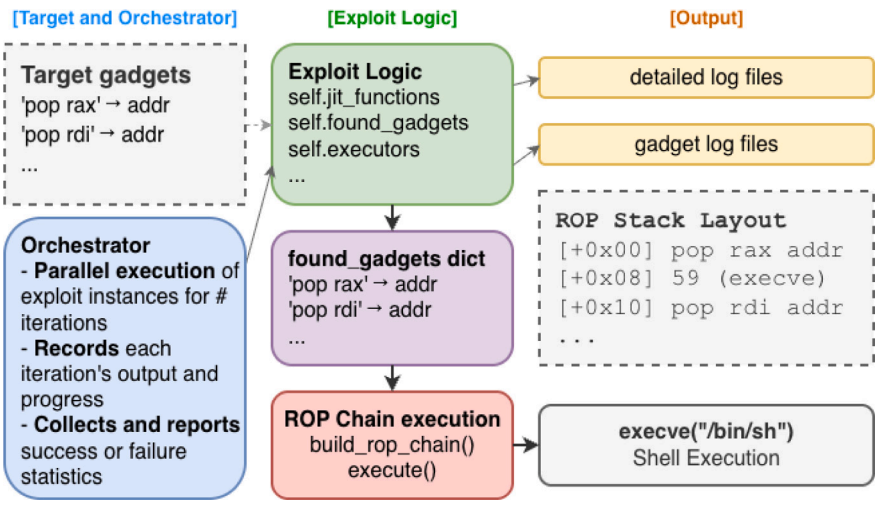


Fig. 2. StencilLeak attack workflow.

Table 2
Static gadget coverage across 274 stencil templates (aligned scan only).

Gadget category	Count (templates)	Remarks
`total ret/tail ret`	90/4	Found in only 4/274 stencils (1.5%); ret appears elsewhere in code
`pop rbx; ret`	0	Not found in static analysis (may appear via unaligned decoding)
`pop rdi; ret`	1	Only found in `shim`
`pop rcx; ret`	0	Not found in static analysis
`xor edx, edx`	7	Zeroing helpers for counters/flags
`pop rax; ret` , `pop rsi; ret` , `pop rdx; ret`	0	Never emitted; prevents fully self-contained chains
`syscall` , `mov eax, 59`	0	Syscall setup helpers
`jmp rel32`, `jne rel32`, `je rel32`	>1000	Abundant indirect control flow
`call rel32`	91	Helper dispatch sequences

Table 3
Hardware & software settings.

Property	Value
Platform	Intel Xeon 3 GHz (Cascadelake)
OS	Ubuntu 24.04.2 LTS
Kernel	6.8.0
ASLR	Enabled
AppArmor	Enforced
CPython	3.14 (official source)
Build options	`--enable-experimental-jit --enable-optimizations`
Privilege	Not required (normal user)

Table 4
Runtime gadget impact of user-level code patterns (100 runs per pattern, averaged over runs)

Pattern	Gadgets	vs. Baseline
Basic loop (baseline)	2600	–
Multi-argument function calls	10,200	+292%
Nested functions/closures	7200	+177%
Exception handling-heavy	2700	+4%
Container operations	10,060	+287%
Object attribute access	6000	+131%
Unpacking (`*args`)	5000	+92%
Recursion-heavy	4500	+73%

`ret`, `pop rsi; ret`, `pop rdx; ret`, `syscall`, or `mov eax, 59`. At the same time, control-flow primitives (`jmp rel32`, `jne rel32`, `je rel32`, `call rel32`) appear more than a thousand times across the templates, but none with register pop or syscall sequences. **This static decode analysis highlights that templated emitters naturally produce register-pop and syscall gadgets very rarely**; these gaps must be filled by other code regions or unaligned decoding once we move beyond template-only analysis.

User-level pattern impact. Beyond this static view, we craft Python functions with diverse control-flow motifs to see how user code steers runtime output. Table 4 shows that multi-argument calls (+292%), container operations (+287%), and nested closures (+177%) markedly increase gadget counts, **which means that certain Python code patterns generate far more gadgets**. It is because they drive register/stack pressure. Exception-heavy code shows minimal increase (+4%). These observations motivate the fixed high-yield template fed to the JITFunctionGenerator, which we reuse for the runtime experiments that follow.

JITFunctionGenerator setup. The JITFunctionGenerator module composes Python functions from these motifs (e.g., multi-argument calls, recursion, nested closures, container-heavy loops) and drives them through a 6000-iteration warmup harness. A representative high-yield template combines these constructs to ensure that the generated Python code includes the most gadget-rich configurations a user-crafted script can induce.

5.3. Experiment 2: Gadget catalog per stencil

Building on the template-level catalog in Experiment 1, this experiment quantifies how many gadget types each stencil contributes after dynamic patching. We generate 100 optimized functions per run with the JITFunctionGenerator and repeat this process for three separate runs, capturing both pre- and post-patch executor buffers. For each capture we (1) dump `code_body[]` and `data_body[]` around every `patch_*` call, (2) disassemble both snapshots, and (3) identify gadgets by scanning for byte patterns and verifying candidates via

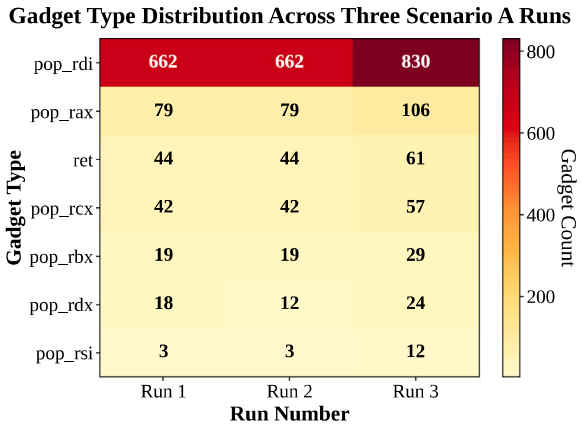


Fig. 3. Gadget catalog showing per-run gadget counts and the dominance of `pop_rdi` templates across all three captures.

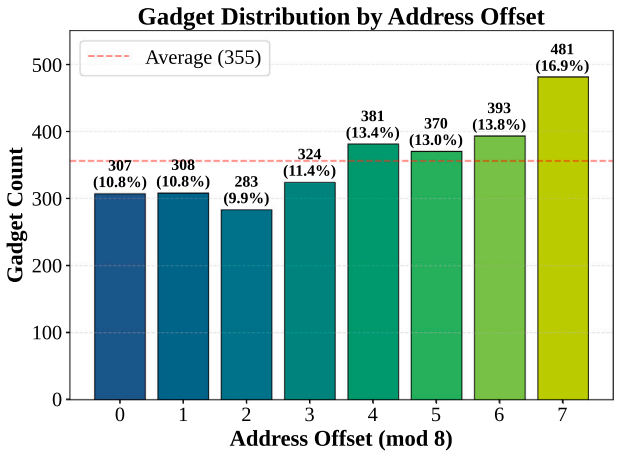


Fig. 4. Experiment 3 byte-shifted decoding: starting at byte offsets 4–7 reveals most unintended gadgets; offset 7 alone captures 16.9% of the total.

disassembly. The aggregated dataset backs a heat map of stencil IDs versus gadget categories and highlights how runtime patching enriches or shifts the template-level gadget pool.

Fig. 3 summarizes the results of three runs (867, 861, and 1119 gadgets respectively), yielding 949 ± 147 gadgets on average. The type distribution is quite skewed: `pop_rdi` accounts for 75.7% of all gadgets (662–830 instances per run), followed by `pop_rax` at 9.3% (86–93 instances) and `ret` at 5.2%. Less frequent but still present are `pop_rcx` (5.0%), `pop_rbx` (2.4%), `pop_rdx` (1.9%), and `pop_rsi` (0.6%). Because the pre- and post-patch counts are identical for every stencil, the structural easy gadgets (`pop rsi`/`pop rdi`) counted here come from the fixed stencil templates rather than the patch routines. This equality does not, however, imply that patching creates no gadgets: the bottleneck gadgets `pop rax; ret`/`pop rdx; ret` and `syscall` are not template bytes at all but are patch-borne, forming only at runtime, and are analyzed separately in Section 5.5. The `pop_rax`/`pop_rdx` tallies above count the bare aligned opcodes, not these `ret`-terminated gadgets, which an aligned per-stencil count never registers; this is why the gadgets are invisible to this catalog yet surface under the byte-shifted decoding of Experiment 3; the 370,400 runtime patch calls, tallied by that same aligned count, do not contradict this.

5.4. Experiment 3: Byte-shifted (unaligned) decoding

Because x86_64 instructions have variable lengths, useful byte patterns (gadgets) can appear even when the decoder starts across aligned instruction boundaries or within freshly patched regions. We refer to this practice of rescanning the same buffer at byte offsets 0–7 as *byte-shifted decoding*. To quantify the effect, we scan for gadget byte patterns at each offset on every captured buffer and count gadgets before/after patching, normalized by region size. Fig. 4 highlights which offsets (typically 4–7, peaking at offset 7, where the 0xC3/0xFF anchor bytes fall) maximize gadget density.

Byte-shifted decoding shows that unaligned views expose significantly more gadgets (Fig. 4). Starting the decoder at byte offsets 4–7 produces 13.4%–16.9% of the gadgets (versus the 12.5% uniform baseline), whereas offsets 0–1 never exceed 11%. Offset 7 peaks at 481 gadgets (16.9%), which is 70% more than the weakest offsets. The gadgets that byte-shifted decoding surfaces are not inside the static template bytes but inside the *runtime-patched immediates*: specifically the `0f 05` bytes from runtime pointer immediates decoded unaligned (per-process ASLR-dependent; Section 5.5) and the `58 c3`/`5a c3` ASLR-randomized runtime heap pointers the JIT patches into `movabs` immediates (decoded from an immediate's low byte). Because these bytes are engine- or allocator-controlled and patched into the buffer at runtime, an operand-value sweep of attacker-chosen constants is uninformative; the variables that determine gadget yield are which stencils are emitted and the process's ASLR layout.

5.5. Experiment 4: Patch impact analysis

This subsection analyzes how the copy-and-patch step's *runtime patching* shapes the emitted gadget surface. The bottleneck gadgets `pop rax` and `pop rdx` are not deterministic features of any stencil; unlike the fixed structural gadgets `pop rsi`/`pop rdi`, they are created by that runtime patching and surface, under unaligned decoding, only for particular per-process ASLR layouts.

Origin-analysis experiment. To trace each gadget byte back to the instruction that carries it and classify what its immediate points to, we generated five branch- and integer-arithmetic-dense synthetic functions (each a `range(48)` loop of 25–55 conditional branches over integer operations) and invoked each 1500 times so the JIT compiled them into hot-trace executors (16 in total). Disassembling the emitted machine code, we analyzed two things: (i) whether the gadget byte pairs `58 c3`/`5a c3`/`0f 05`, absent from every static template (`jit_stencils.h`), appear in the patched runtime code; and (ii) what each 8-byte `movabs` immediate the JIT patches into the code points to (a program constant object, identified from its PyObject header, or an engine structure, via a `dladdr` symbol lookup). The generator is:

```
def make(idx):
    r = random.Random(idx)
    L = ["def _f%d(n):" % idx, "  s = 0", "  for i in range(n):"]
    for _ in range(r.randint(25, 55)): # 25-55 branch
         blocks
        L.append("    if (i ^ %d) & %d:" % (r.randint(1, 9999),
            r.choice([1, 2, 3, 7])))
        L.append("      s += (i ^ %d)" % r.randint(1, 10 ** r.
            randint(2, 7)))
        L.append("    else:")
        L.append("      s -= (i ^ %d)" % r.randint(1, 9999))
    L.append("  if s > 100000: s -= 99999")
    L.append("  return s")
    ns = {}; exec("\n".join(L), ns); return ns["_f%d" % idx]
```

Results. For (i), the three byte pairs occur in no static template yet appear in the emitted executors, confirming that they are created dynamically by patching. For (ii), of the 4007 `movabs` immediates classified (this is the *whole* patched-immediate pool, not only gadget-forming ones), only 90 (about 2%) point to program constants; the

Table 5
`movabs` immediate classification: which pointer each patched immediate holds (all patched immediates, not only gadget-forming ones).

Immediate target	Count	Share	Emitting role
Heap/other runtime pointer	2365	59.0%	Anonymous allocator-controlled heap/data pointers (gadget-bearing family)
Executor pointer (`uop_executor`)	1018	25.4%	Exit/resume targets (`_EXIT_TRACE`, `_DEOPT`, `_SET_IP`)
`_PyRuntime`	266	6.6%	Engine global
`PyLong_Type`	119	3.0%	Guard type check (`_GUARD_*`)
`_Py_TrueStruct` (True)	91	2.3%	Engine singleton
Program constant (`_LOAD_CONST_INLINE`)	90	2.2%	The program's own constants
`_Py_FalseStruct` (False)	55	1.4%	Engine singleton
`PyRangeIter_Type`	3	0.1%	Guard type check
Total	**4007**	**100%**	

Table 6
Micro-op frequency across the same 16 executors (12 representative high-frequency micro-ops).

Micro-op	Count	Role
`_DEOPT`	467	Bail out to the interpreter (exit-target pointer)
`_SET_IP`	315	Mark where to resume (code pointer)
`_CHECK_VALIDITY`	311	Confirm the trace is still valid
`_EXIT_TRACE`	165	Exit to another executor (executor pointer)
`_GUARD_BINARY_OP_EXTEND`	156	Type/shape guard before the op
`_BINARY_OP_EXTEND`	156	The arithmetic operation itself
`_LOAD_FAST_BORROW_2`	103	Load a local
`_LOAD_CONST_INLINE`	90	Load a program constant (pointer)
`_ERROR_POP_N`	56	Error-path cleanup
`_TO_BOOL_INT`	53	Branch condition
`_GUARD_TOS_INT`	50	Guard the top of stack is an int
`_GUARD_IS_TRUE_POP`	38	Branch guard

other 98% are engine- and allocator-controlled runtime pointers of several kinds (the largest group anonymous heap and data pointers, the rest executor and exit/resume pointers, engine globals, types, and singletons). Separately, of the immediates whose low bytes actually form a `58 c3`/`5a c3` gadget, every one fell in the anonymous heap group, an allocator-controlled runtime pointer (no live constant object). Table 5 gives the full classification, and Table 6 the uop frequencies that explain why a denser trace emits more such pointer immediates.

Origin of the bottleneck gadgets. The `pop rax` (`0x58`) and `pop rdx` (`0x5a`) bytes do occur as aligned structural stack-management opcodes inside stencils (roughly 27 `pop rax` per executor on average), but at runtime each is followed by the stencil's own continuation rather than a `ret`, so none forms a `pop; ret` gadget. The usable `58 c3` and `5a c3` gadgets occur in no static template (aligned or unaligned) and at no concatenation seam; disassembly shows they materialize only at runtime, when the JIT patches an ASLR-randomized *runtime pointer* into a `movabs` immediate whose own low bytes happen to spell `58 c3`/`5a c3` (the byte-level mechanism is detailed next). As the classification above shows, these gadget-forming immediates are allocator-controlled anonymous heap pointers (which uop emits them we do not establish), so their presence is a per-process ASLR coincidence, which is why `pop rax`/`pop rdx` are probabilistic: each compiled function emits many such `movabs` pointer immediates, any single one rarely matches, and across 500 analyzed attempts their per-attempt availability is a measured 72.4%/65.0% (the rate at which the time-budgeted search found them, not raw byte presence).

How the gadget forms. The gadget is two *consecutive* bytes *within* the 8-byte pointer immediate, not a template byte followed by a separate `ret`: read at an unaligned offset, one byte of the immediate decodes as `pop rax`/`pop rdx` (`0x58`/`0x5a`) and the next byte, when it is `0xc3`, as `ret`, so the trailing `0xc3` is itself a byte of the patched pointer rather than a following instruction. Across 1642 recovered gadgets the pattern occurs only at the immediate's two lowest byte-positions (position 0 in 42% and position 1 in 58% of `pop rax` cases) and never higher, because the pointer's upper bytes are the ASLR region base (typical userspace values such as `0x7f`/`0x5c`/`0x61`) that cannot equal `0xc3`. Dumping every patched immediate across two processes with different ASLR bases confirms the byte structure: for a *single* pointer, bits 0–11 (the 4 KiB page offset) are fixed while bits 12 and above are ASLR-randomized, which identifies each immediate as a genuine runtime pointer. The pattern match, however, is over the *many different* candidate pointers a trace emits, whose low bytes vary from allocation to allocation and from process to process; forming it requires only a few of those runtime-varying bits to align (of order 1/16 per candidate immediate; an order-of-magnitude estimate), while the fixed high bytes (the address's `0x7f`-type top, above the pattern) play no role, which is why the pattern appears only at these two low positions and never deeper. This is precisely *why* the bottleneck gadgets are probabilistic: whether a pointer's low bytes spell the pattern is decided by the specific runtime pointer values a trace emits, the allocator's placement together with the process's ASLR base, not by any stencil, so the attacker can neither predict nor choose them. Because each trace emits many such pointer immediates, per-attempt availability follows $1-(1-p)^N$ in the number N of candidate immediates; with $p \approx 1/16$ and, for a general campaign function, $N \approx 20$, this predicts about 73%, close to the 72.4% measured on that campaign workload. The branch- and arithmetic-dense functions of this experiment emit many more such immediates (the classification above), for which the same model approaches certainty; what a workload changes is N, not p.

Workload influence. Because these gadgets ride on runtime-patched pointer immediates rather than on specific stencils, the input workload cannot deterministically trigger a given gadget; a branch- and arithmetic-heavy function raises only the *number* of pointer immediates a trace emits, and hence the per-attempt *probability* that some pointer's low bytes match. Success probability is thus shaped by, not deterministically fixed by, the input structure.

5.6. Experiment 5: Harness-assisted construction and execution of a JIT-resident ROP chain

This experiment tests whether gadgets found in emitted JIT executor buffers can be assembled into and executed as a complete `execve(''/bin/sh'')` ROP chain after the disclosure, writable chain storage, crafted stack, and control transfer to the first gadget assumed in Section 4.1 have been supplied. It does not identify or exploit a memory-safety vulnerability, and it does not independently obtain instruction-pointer or stack control.

Table 7
ROP chain gadget addresses (Example successful run, 4.6s; offsets relative to JIT base; unaligned decode).

Gadget	Offset	min. Discovery time
`pop rsi; ret`	+0 × 001d	~0.5 s (1st)
`pop rdi; ret`	+0 × 001f	~0.5 s (1st)
`pop rdx; ret`	+0 × 0165	~0.5 s (1st)
`pop rax; ret`	+0 × 008e	~2 s (2nd)
`syscall`	+0 × 0280	~4.6 s (3rd)

Listing 1: Exploit execution output from a successful demonstration (4.6 seconds)

```
$ python3 jitleak_rop.py --count 3000

[*] JIT region base: 0x7f84e7dfa000
[*] Scanning for gadgets...
    func_0: Found pop_rsi @ +0x001d (0.5s)
    func_0: Found pop_rdi @ +0x001f (0.5s)
    func_0: Found pop_rdx @ +0x0165 (0.5s)
    func_3: Found pop_rax @ +0x008e (2.0s)
    func_7: Found syscall @ +0x0280 (4.6s)
[+] All 5 gadgets found! (8 functions scanned)

[*] Building gadget chain...
  [Stack +0x00] = pop rax; ret
  [Stack +0x08] = 0x3b (execve)
  [Stack +0x10] = pop rdi; ret
  [Stack +0x18] = "/bin/sh" address
  [Stack +0x20] = pop rsi; ret
  [Stack +0x28] = 0x0 (NULL)
  [Stack +0x30] = pop rdx; ret
  [Stack +0x38] = 0x0 (NULL)
  [Stack +0x40] = syscall

[*] Executing gadget chain...
$ whoami
researcher
$ exit
[+] Total time: 4.6 seconds
```

The harness uses `_opcode.get_executor()` to obtain executor objects and `ctypes` to read their private fields and emitted JIT bytes. After locating all required gadget patterns, it uses `ctypes` to allocate writable memory and fill it with `/bin/sh` and a stack containing the gadget addresses and operands for the chain. To simulate the assumed stack control and initial transfer, it writes a short trampoline into a separate RWX mapping and invokes that mapping through `CFUNCTYPE`; the trampoline loads `rsp` with the prepared stack address and jumps to the first JIT-resident gadget.

Attack demonstration. Table 7 lists the gadget offsets and discovery times from a successful demonstration (4.6 s total). The offsets are relative to the JIT region base address. Listing 1 shows the complete exploit output, culminating in a native shell prompt.

From that first gadget onward, the `ret`-based chain uses gadget addresses in the JIT's executable mappings and reaches the final `syscall`. The experiment therefore establishes complete chain construction and execution after the stated prerequisites have been supplied; it does not establish the prerequisite memory error or obtain instruction-pointer and stack control by exploiting one. The gadget chain itself reuses RX JIT bytes, but the complete harness does not satisfy strict W^X because the synthetic launcher is stored in an RWX mapping. This result should not be interpreted as an end-to-end memory-corruption exploit, a demonstrated bypass of the modeled Python-interface policy, or a seccomp or container escape.

Table 8
Large-scale attack configuration (success = found all gadgets within 2 h of search).

Parameter	Value
Total attempts	500
Functions per attempt	3000 JIT-compiled
Timeout per attempt	2 h (7200 s)
Parallel workers	8
Aggregate time over successful runs	~211 h

Table 9
ROP chaining time statistics (251/500 attempts).

Metric	Time
Fastest success	4.6 s
Slowest success	117.9 min (7074.8 s)
Average	50.4 min (3026.8 s)
Median	43.0 min (2578.2 s)
Std. Dev.	33.7 min (2024.5 s)

5.7. Experiment 6: Attack feasibility at scale

5.7.1. Attack success rates

Each attempt regenerates 3000 functions by JITFunctionGenerator and produces a fresh capture. The workload strategy (e.g., mixing recursive calls and dictionary operations) and the specific Python functions generated are fixed across attempts, since the JITFunctionGenerator is deterministic and unseeded; the functions therefore do not vary from attempt to attempt. Thus, success depends on whether each attempt's per-process ASLR layout exposes the bottleneck `pop rax`/`pop rdx` pattern in the low bytes of the runtime-patched pointer immediates the workload emits (Section 5.5), not on triggering any specific "bottleneck stencil". Table 8 summarizes the large-scale attack parameters.

Under the default setting where Garbage Collection (GC) is enabled, the attack success rate is 36.2%. Disabling GC increases the success rate to 50.2% (251 successes) in our 500-attempt evaluation. This improvement occurs because disabling GC prevents the reclamation of unused executor buffers; when GC is active, buffers containing potential gadgets may be released before the full chain is constructed, forcing the attacker to restart the search. Each successful run located all five required gadget patterns (a gadget-set discovery in this dry-run benchmark, not a native-payload execution), while the remaining 249 attempts (49.8%) timed out. Table 9 presents the timing statistics for these successful attempts.

Per-attempt gadget-set discovery is 50.2%. Under an unverified iid Bernoulli assumption, the modeled cumulative probability of at least one gadget-complete run rises with repetition: 75.2% after 2 tries, 87.6% after 3, 93.8% after 4, 96.9% after 5, 99.2% after 7, and 99.9% after 10 (model-derived estimates, not observed retry rates; see the benchmark interpretation in Section 5.7). In settings with repeated access (web services, multi-tenant clouds), repeated attempts correspondingly raise the modeled probability of locating the full gadget set.

5.7.2. Gadget difficulty analysis

Our statistics on gadget discovery reveal a striking disparity in difficulty among the five gadgets, summarized in Table 10. Three gadgets (`pop rsi`, `pop rdi`, `syscall`) are found in nearly every attempt within seconds (with `syscall` emerging within the first 10 functions in 99.6% of cases), while two gadgets (`pop rdx`, `pop rax`) remain the exclusive bottlenecks.

Table 10
Individual gadget discovery statistics (500 attempts).

Gadget	Found	Avg time	Max time	Difficulty
`pop rsi`	100.0%	0.5 s	<2 s	Very easy
`pop rdi`	100.0%	0.5 s	<2 s	Very easy
`syscall`	99.6%	4.5 s	71.2 s	Moderate (~8–10 funcs)
`pop rax`	72.4%	1655 s	7074 s	Bottleneck
`pop rdx`	65.0%	2634 s	6151 s	Bottleneck

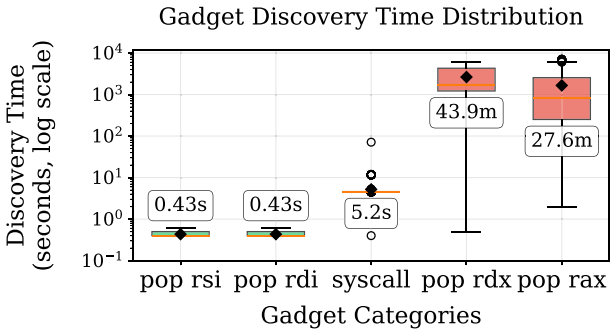


Fig. 5. Box-and-whisker distribution of per-gadget discovery times across 500 attempts (log time axis). The very easy gadgets (`pop rsi/rdi`) cluster at sub-second latencies with minimal spread (found in 100% of runs), while `syscall` stays near-certain but Moderate (99.6%, about 4.5s on average, up to 71.2s). Bottleneck gadgets (`pop rax/rdx`) show wide, long-tailed distributions and partial availability (72.4%/65.0% of runs), extending to ~7000 s.

The difficulty split in the discovery table tracks each gadget's origin class rather than chance. The very easy gadgets, `pop rsi` and `pop rdi`, are deterministic structural template bytes present in essentially every executor, so a search finds them almost immediately, within about half a second in nearly 100% of runs. The bottleneck gadgets `pop rax` and `pop rdx` are patch-borne instead: as Section 5.5 establishes, they are the low bytes of ASLR-randomized runtime-patched `movabs` pointer immediates, so they surface only at unaligned offsets and only for the particular per-process ASLR layouts whose pointers happen to spell the pattern. Whether any given attempt exposes them is therefore probabilistic, which is why their discovery is long-tailed and only partial, found in 72.4% and 65.0% of runs over 20 to 70 min, and why they are absent from the aligned static scans of Experiment 1. `syscall` is patch-borne as well, yet it stays near-certain (99.6%, Moderate), so a shared origin does not by itself set difficulty: its availability is not gated by the same rare per-pointer low-byte match that throttles `pop rax` and `pop rdx`. The byte-level mechanism behind this probabilistic availability, and behind `syscall`'s different behavior, is derived in Section 5.5.

Key Findings on Gadget Discovery: (1) Full gadget discovery rate stabilizes at 50.2% per attempt; (2) `pop rax` and `pop rdx` remain the bottleneck gadgets, together accounting for 100% of last-found gadgets (approximately 26% vs. 74% respectively); (3) Easy gadgets appear within the first few functions, while bottleneck gadgets often take 20–70 min to find, explaining the bimodal completion time. Fig. 5 plots per-gadget discovery times, highlighting the gap between always-present, sub-second gadgets and the long-tailed bottlenecks.

5.7.3. Probabilistic attack feasibility

Benchmark interpretation. In 500 fresh CPython processes, each using the same deterministic 3000-function workload from our fixed `JITFunctionGenerator` (varied functions produced by a fixed generator applied identically to every run and not hand-tuned per run), GC disabled, and a two-hour search budget, 251 dry runs located all five required gadget patterns (50.2%; 95% Wilson confidence interval approximately 45.8%–54.6%). This is a gadget-set discovery rate for the stated configuration, not a payload-execution rate or an intrinsic success probability of CPython's JIT. Fresh processes provide distinct trials, but trial independence was not empirically tested; the Wilson interval above and the projection that follows therefore both rest on an unverified iid binomial assumption. If one models the trials as iid Bernoulli draws with $p = 0.502$, the probability of at least one gadget-complete run in seven trials is $1 - (1 - 0.502)^7 = 99.2\%$. The 99.2% figure is therefore a model-derived estimate, not a directly observed retry rate. The present benchmark does not isolate garbage collection or OS scheduling as causal sources of variation; GC was disabled, and scheduling is only a possible contributor to elapsed-time variance and timeouts under parallel execution. The build, hardware, and OS are those reported in Section 5.1: CPython 3.14 with `--enable-experimental-jit--enable-optimizations`, an Intel Xeon Cascadelake host, Ubuntu 24.04 (kernel 6.8), and ASLR enabled. Each attempt warms the Tier-2 JIT (about 5000 iterations) over a deterministic generator in which function i carries $10,000+1,000i$ loop iterations across a fixed pattern set (no random seed); the 500 attempts run in parallel with one worker per core under a per-attempt two-hour (7,200 s) timeout. The 249 non-successful attempts are all timeouts: the budget elapsed before all five gadgets were located, which for almost all such processes means the per-process ASLR layout did not expose the bottleneck `pop rax/pop rdx` pattern within the functions searched (`syscall`, present in 99.6% of runs, could account for at most a couple), rather than a separately classified gadget-absent outcome. Because this generator spreads across many gadget types rather than targeting the two bottleneck gadgets (`pop rax/pop rdx`), 50.2% is the rate of an unbiased general search; a workload biased toward those two gadgets would raise their availability but not necessarily the five-gadget rate, since it does not raise `syscall`, whose availability can fall.

Table 11
Gadget discovery statistics across 500 attempts. Lower function index means faster discovery.

Gadget	Discovery rate	Mean func. index	Median	Std Dev
`pop rdi`	100.00%	0.0	0	0.0
`pop rsi`	100.00%	0.0	0	0.0
`syscall`	99.60%	7.6	7	2.8
`pop rax`	72.40%	232.5	192	152.9
`pop rdx`	65.00%	325.9	278	133.7

Beyond the across-attempt retry projection above, we now examine whether the five gadget discoveries *within* a single run are independent events or exhibit statistical correlation. A successful StencilLeak attack requires finding all five gadgets: `pop rax`, `pop rdi`, `pop rdx`, `pop rsi`, and `syscall`. Table 11 summarizes when each gadget first appeared (the *function index*).

Gadget correlation analysis. If gadget discoveries were independent, multiplying their probabilities would predict:

$$P_{\text{theory}} = P(\text{rax}) \times P(\text{rdi}) \times P(\text{rdx}) \times P(\text{rsi}) \times P(\text{syscall})$$
$$= 0.724 \times 1.0 \times 0.650 \times 1.0 \times 0.996 \approx 46.9\%$$

Yet we observed 50.2% (251/500), notably higher. This discrepancy suggests the bottleneck gadgets are correlated.

A Chi-squared test confirmed this: $\chi^2 = 12.27$ far exceeds the critical value of 3.84 ($\alpha = 0.05$). When `pop rax` appears, `pop rdx` is more likely to appear as well. Quantitatively, among the 362 attempts that found `pop rax`, 252 also found `pop rdx`:

$$P(\text{rdx}|\text{rax}) = \frac{252}{362} = 69.6\%$$

This exceeds the marginal $P(\text{rdx}) = 65.0\%$, confirming positive correlation.

Using this conditional dependency, the refined model predicts:

$$P_{\text{refined}} = P(\text{rax}) \times P(\text{rdx}|\text{rax}) \times P(\text{rdi}) \times P(\text{rsi}) \times P(\text{syscall})$$

$$= 0.724 \times 0.696 \times 1.0 \times 1.0 \times 0.996 = 50.2\%$$

This reconstruction recovers the observed 50.2% as a consistency check rather than independent confirmation, since $P(\text{rax})\,P(\text{rdx} \mid \text{rax})$ is by definition the joint rate and $P(\text{rdx} \mid \text{rax})$ was measured on the same runs; the positive co-occurrence itself is established by the chi-squared result and by $P(\text{rdx} \mid \text{rax}) = 69.6\%$ exceeding the marginal $P(\text{rdx}) = 65.0\%$ above. **Verified explanation.** The positive co-occurrence of `pop rax` and `pop rdx` gadgets arises because a process's runtime heap allocations share the same heap region, rather than because both gadgets occur in the same stencil. A per-process ASLR layout that exposes the `58 c3` or `5a c3` pattern in the low bytes of one such heap pointer tends to expose it across multiple heap pointers. This shared-region effect is the origin of the positive `pop rax`/`pop rdx` co-occurrence within a single run rather than any single stencil containing both gadgets.

Attack feasibility. With a 50.2% per-attempt success rate, an attacker needs ~2 attempts on average, and based on the cumulative probability of independent Bernoulli trials, 5 attempts suffice for 95% confidence. The presence of time outliers in Fig. 5 additionally suggests that in real-world noisy environments, OS thread scheduling and unpredictable Garbage Collector (GC) pauses may introduce latency spikes, slightly degrading the absolute success speed. As for *workload shaping*: because `pop rax`/`pop rdx` are the low bytes of ASLR-randomized runtime pointers, it does not *deterministically* trigger them. What the workload actually controls is *how many* `movabs` pointer immediates a trace emits, and (Section 5.5) these gadgets ride on those runtime-patched, engine- and allocator-controlled pointer immediates, whose count scales with a trace's branch and typed-operation density (the guard, control, and typed-operation uops that dominate `movabs` emission grow with it), not with its constants. A denser workload thus raises the *number* of per-process ASLR opportunities and hence the per-attempt *probability* of a match, so a denser workload would be expected to raise the availability of `pop rax`/`pop rdx` above the 72.4%/65.0% measured on the general benchmark generator. The address bytes, however, stay engine/allocator- and ASLR-controlled, never attacker-placed; and the same shaping does *not* lift `syscall`, whose availability is not governed by the *number* of these pointers. Because shaping cannot lift `syscall`, it raises the availability of the `pop rax`/`pop rdx` bottlenecks specifically and does not push whole-chain success toward certainty. In environments with persistent access (web services, cloud APIs, containerized workloads), StencilLeak attack is a practical threat. Our results show that enabling the JIT expands the process's code-reuse attack surface; the following section discusses mitigation and the challenge of preserving JIT performance.

6. Discussion

6.1. Prerequisites for reliable exploitation

JIT Base Disclosure. Reliable StencilLeak exploitation requires bypassing ASLR to determine absolute gadget addresses. Consistent with the threat model (Section 4.1), this disclosure is supplied by the assumed independent memory-safety vulnerability, not by a JIT introspection API; the `get_executor()` and `/proc/self/maps` interfaces that expose the base in standard builds are used only by our evaluation harness. Given such a disclosure, the attacker observes the runtime addresses and patched bytes that ASLR would otherwise withhold and can chain the discovered gadgets.

Concrete instantiation of the assumed capabilities. The disclosure, writable-storage, and control-transfer capabilities our threat model assumes are not merely postulated: on the same JIT-enabled build, an already-patched CPython use-after-free realizes them. CVE-2026-6100, in the `lzma`/`bz2` decompressors, yields an arbitrary read, an arbitrary write, and a control-flow transfer to an address of our choosing, with every address we need obtained for free from Python's `id()` rather than from a separate information leak. It does not, by itself, provide the controlled stack that a full ROP launch would additionally require.

Concretely, the bug is a dangling input cursor: when an allocation fails partway through decompression, the decompressor keeps a pointer into a freed buffer, and the next decompression writes through it. We groom the heap, arranging the order and sizes of allocations so that the freed buffer is reused by a large Python `list` rather than by unrelated data, and keep the reuse write header-preserving, so it lands cleanly on the list's element pointers (a type confusion) rather than crashing. Two fake objects then supply the primitives: a fake `bytearray` placed at a chosen address turns ordinary `list` access into an arbitrary read and write of that address, and a fake type whose `tp_call` slot holds a chosen address transfers control there when the confused object is called.

With these primitives we reach the JIT state we characterize. Using only the arbitrary read, we recover a live Tier-2 executor's `size` and `jit_addr` fields and the first bytes of its emitted machine code, and in the same run transfer control from that disclosure to a chosen address. To confirm the read is genuine, we compare it against a `ctypes` oracle (the same executor fields the harness otherwise reads directly): across three independent targets, the recovered `jit_addr`, `size`, and machine-code prolog match byte for byte, and a `SIGSEGV` handler confirms the faulting instruction pointer equals the chosen address. The integrated leak-then-transfer check reproduced at 6/6 runs with no grooming failure.

The demonstration transfers control only to a benign unmapped marker; it does not mine gadgets, assemble a ROP chain, or launch a shell. Its preconditions and limits are explicit. (1) The bug fires only when the decompressor's tail-copy allocation fails, that is, under an out-of-memory condition at that specific call; in the demonstration we make this failure deterministic with a fault-injection tool, and we do not measure how often it is reached under real, uninstrumented memory pressure. (2) The heap grooming is sensitive to allocation order, so an unrelated allocation can shift the layout and miss. (3) The single heap-header read used for consistency is an ordinary `/proc/self/mem` access, not a native-introspection API such as `ctypes`. (4) The free `id()` leaks assume the attacker runs Python in the process; this holds for the source-influencing tenant, including the UDF and hosted-notebook settings of Section 4.1, but not for a remote, input-only attacker. We therefore treat CVE-2026-6100 as evidence that the disclosure, write, and control-transfer components of the assumed prerequisite class are concretely realizable, not as an unassisted end-to-end exploit. The primitive-construction and verification scripts, the step-by-step findings (R0 through R6), and the reproducibility record are provided as supplementary material at https://doi.org/10.5281/zenodo.23569801.

6.2. Differentiating from data-centric JIT spraying

Traditional JIT Spraying relies on injecting attacker-controlled data (e.g., large constants or immediate values) directly into the executable instruction stream, which the processor then misinterprets as instructions via unaligned decoding. In CPython's copy-and-patch JIT, an attacker-chosen constant's raw *value* bytes do not enter the stream: they remain in an out-of-line `co_consts` object. One group of uops could in principle expose a source-controlled operand: the index-writing `_LOAD_CONST_MORTAL`/`_IMMORTAL` write the `co_consts` index into the 8-byte immediate of a `movabsq $imm64,%rcx` (`patch_64(code+0xe, oparg)`), where the following `movzwl %cx,%ecx` masks only the value used for indexing, not the physical bytes, so a crafted 16-bit index such as `0xc35a` would place `5a c3` (`pop rdx; ret`) inline; we therefore do not claim the architecture categorically forbids chosen-immediate spraying. In practice, however, the Tier-2 optimizer's abstract-interpreter pass (`Python/optimizer_`

Table 12
Origins of the three JIT-resident gadget classes.

Gadget	Bytes	Origin	Determinism (rate)
`pop rsi/rdi`	`5e../5f c3`	shim callee-saved epilogue; fixed structural opcode at +0 × 1d/+0 × 1f	deterministic (~100%)
`syscall`	`0f 05`	decoded unaligned from a runtime-patched `movabs` pointer immediate (higher byte position); not a static stencil byte	per-process ASLR (99.6% in campaign, workload-dependent)
`pop rax/rdx`	`58 c3/5a c3`	low two bytes of an ASLR-randomized runtime pointer `movabs` immediate (not a constant object)	per-process ASLR (72.4%/65.0%)

`bytecodes.c`) *unconditionally* rewrites every constant load to a pointer-writing uop (`_LOAD_CONST_INLINE/_BORROW`, operand `(uintptr_t)val`) and so never emits an index-writing uop (verified in the optimizer source and across hundreds of JIT-compiled functions), so this vector is not realized. The gadgets characterized in this work come instead from the engine's structural opcodes and allocator-controlled runtime heap pointers, as described next.

Instead, StencilLeak's gadgets are engine- and allocator-controlled bytes exposed by unaligned decoding, arising from two structural mechanisms (fixed structural opcodes, and the low bytes of runtime-patched `movabs` pointer immediates) as three gadget types. The always-available *easy* gadgets (`pop rsi`, `pop rdi`) are fixed structural opcodes: the shim's callee-saved-register restore epilogue `...41 5e 41 5f c3`, present in every executor at a stable offset. The `syscall` gadget, by contrast, is a runtime-patched `movabs` pointer immediate decoded unaligned rather than a static template byte, and is *probabilistic* like the bottleneck gadgets, though its `0f 05` sits at a higher byte position (carrying no `0xc3`, it is not confined to the immediate's lowest bytes as the `pop` pattern is), so its per-run availability (99.6% in our campaign) is workload-dependent. The bottleneck gadgets `pop rax; ret` (`58 c3`) and `pop rdx; ret` (`5a c3`) are *probabilistic*: they are the low two bytes of an ASLR-randomized runtime pointer that the JIT patches into a `movabs` immediate (an allocator/engine-controlled pointer, not the constant object itself), and surface only for particular per-process ASLR layouts: any single such pointer matches rarely, but each attempt emits many pointer immediates, and across 500 attempts the per-attempt availability of these gadgets is a measured 72.4%/65.0%. None of these three pairs occurs in any static template (aligned or unaligned) or at any concatenation seam, and the abundant *structural* `popq %rax` inside arithmetic stencils never forms a gadget, because it is followed by the stencil's continuation, not a `ret`. Because these gadget bytes are engine- or allocator-controlled rather than chosen constant values, constant blinding aimed at attacker-chosen immediates does not directly remove them (blinding chosen *indices* would instead address the constrained index vector noted above); other code-randomization defenses remain a separate question. Two forms must be distinguished: classic *data-value* spraying, where an attacker-chosen constant value is emitted inline and misdecoded, does not apply (the value stays out-of-line), whereas StencilLeak involves a *structural, emission-level* influence, since source structure shapes which stencils are emitted and combined while the reused gadget bytes remain engine- and allocator-controlled (see Table 12).

6.3. Mitigation strategies

We prioritize mitigations that (1) eliminate the bottleneck gadgets we show to be necessary for exploitation, (2) provide runtime detection against large-scale gadget harvesting, and (3) reduce the stability of any leaked JIT base address. The following items are ordered by their impact on the observed 50.2% StencilLeak attack success rate.

Hardware Control-Flow Integrity. Before the availability-ranked items below, we note an orthogonal, transfer-level defense whose effect does not depend on gadget availability. Hardware Control-Flow Integrity directly constrains this attack. On the backward edge, userspace shadow stack (Intel CET (Shanbhogue et al., 2019)) checks return addresses independently of whether the executing `ret` resides in a static binary or JIT-generated code; when enabled, it stops our demonstrated `ret`-based chain. On the forward edge, Intel IBT or ARM BTI constrains indirect `call/jmp` targets to marked landing pads and can reject an indirect launch into an unmarked, mid-instruction gadget. IBT does not itself validate `ret` targets, so these mechanisms address different transfers. We have not evaluated a JOP or COOP alternative. The demonstrated return-chain mechanism therefore requires userspace shadow stack to be disabled and no forward-edge CFI policy to reject the selected launch transfer.

High-Priority: Eliminate Bottleneck Gadgets. The `pop rax`/`pop rdx` bottleneck gadgets are present in only 72.4% and 65.0% of attempts respectively (their per-attempt availability), and together with the probabilistic `syscall` gadget they gate a complete chain, so reducing their availability would impede the StencilLeak attack. As a naive first attempt, one might try a minimal JIT compiler modification intended to break an *aligned* `pop; ret` in the function epilogue:

Listing 2: Minimal modification of JIT compiler to break gadgets

```
// In emit_function_epilogue():
// naive assumption: an aligned "pop rax; ret" is an exploitable
      gadget
// (the paper shows no such ALIGNED gadget exists; see text below)
// GOOD: pop rax; nop; ret (would break an aligned pair, if one
      existed)
if (last_instruction_is_pop()) {
    emit_nop(); // Insert gadget breaker
}
emit_ret();
```

However, this approach is fundamentally limited by the variable-length instruction encoding of x86_64. As demonstrated in Experiment 3, unaligned byte-shifted decoding surfaces thousands of exploitable sequences from the middle of otherwise benign instructions at offsets 1 through 7. The limitation is first one of *efficacy*, not merely cost: the bottleneck and `syscall` gadgets (`58 c3/5a c3/0f 05`) do not reside in the stencil templates at all but form at runtime inside patched `movabs` pointer immediates (Section 5.5), so padding the templates cannot alter those bytes and cannot remove those gadgets regardless of how much padding is added. To reliably eliminate all such sequences, a defender would need to insert `nop` padding between almost every instruction in the stencil templates to break unintended byte patterns. Such extensive padding would categorically destroy the primary advantages of the copy-and-patch JIT, namely its low memory overhead and high instruction cache efficiency. As a result, this mitigation would render the optimization practically useless for performance-critical workloads.

On the choice of target gadget. `syscall` is the functional linchpin, with no substitute: it enters the kernel and invokes OS functionality (e.g., `execve`). It is not, however, a *bottleneck* gadget: the bottleneck gadgets are the rarely-present `pop rax`/`pop rdx` (72.4%/65.0% of runs), whereas `syscall` (the same probabilistic kind) appears in 99.6% of runs under our campaign workload, an availability that is itself workload-dependent. None of the three occurs as an aligned instruction in a stencil; each surfaces only when an engine-controlled, runtime-patched value is decoded unaligned (a direct consequence of variable-length x86_64 encoding), so none can be removed by breaking or padding aligned emissions alone. We recommend targeting both

`syscall` and the bottleneck gadgets with a feasible measure for each: (i) for the rarely-surfacing `pop rax`/`pop rdx` bottleneck, reduce its *availability*; and (ii) for `syscall`, whose workload-dependent availability cannot be relied on to be low, constrain its *effect*, i.e., a syscall-filtering policy such as seccomp-BPF blocks disallowed syscalls (e.g., `execve`) regardless of whether the gadget is present. Because a complete chain requires all of these gadgets, either lever breaks it.

Medium-Priority: Runtime Detection. Another mitigation is to monitor the anomalous JIT compilation patterns indicative of gadget spraying: (1) Alert if more than 100 JIT functions are generated within 1 min; (2) Flag processes that scan executable memory regions repeatedly; (3) Detect repetitive function signatures that suggest automated gadget discovery. However, these heuristics may produce false positives in legitimate high-throughput workloads (e.g., machine learning inference), and attackers can evade detection by spreading gadget discovery over longer time windows.

Enhanced ASLR for JIT Regions. Re-randomize JIT code region base addresses periodically (e.g., every 5 min) to invalidate previously discovered gadget addresses. Combined with sealing internal APIs like `get_executor()`, this would significantly increase attack complexity. Yet, implementing dynamic re-randomization in a highly interactive, object-oriented runtime like Python presents severe engineering challenges. The Python interpreter frequently communicates with the heap, C-API extensions, and internal object pointers. Relocating active JIT code would require pausing the execution state through a stop-the-world mechanism to recursively patch thousands of active runtime pointers. To prevent such a process from incurring an unacceptable latency spike, significant architectural optimization would be required, akin to mitigating severe Garbage Collector pauses. Furthermore, our 500-attempt evaluation demonstrated that the StencilLeak attack can complete in just 4.6 s. To effectively thwart the attack, the re-randomization interval would need to be very short, typically under four seconds. This frequent interruption imposes a continuous performance penalty making it highly impractical for mainstream production deployments.

Prior Mitigations. Existing literature explores several defenses against JIT-ROP and JIT Spraying (alignment enforcement (Yee et al., 2009), removal of unaligned gadgets (Onarlioglu et al., 2010), in-place code randomization (Pappas et al., 2012), JIT constant blinding (Athanasakis et al., 2015), and transparent/runtime code randomization (Homescu et al., 2013; Bigelow et al., 2015)). Constant blinding, however, targets attacker-chosen immediates and so does not apply to StencilLeak's gadgets, which form from engine/allocator-controlled bytes (Section 6.2). Applied to copy-and-patch, the alignment, gadget-removal, and re-randomization families incur code-size, instruction-cache, active-JIT-code relocation, and runtime-reference-update costs that we did not measure, so evaluating their overhead for CPython remains future work.

6.4. Comparison with other JIT designs

The empirical comparison is limited to CPython's experimental copy-and-patch JIT and to the benchmark configuration in Section 5.7. A matched success-rate comparison with other JITs would require implementation-specific attacks, equivalent disclosure and launch primitives, a common payload, and equivalent search budgets. Within the evaluated mechanism, two architectural observations are supported. (1) *Offline knowledge versus runtime observation*: fixed stencil bytes and categories of gadget-forming patch sites can be cataloged offline. Offline knowledge of these stencil bytes and patch-site categories may lower the entropy of gadget discovery: traditional ROP must scan arbitrary target code to identify available gadgets, and a full optimizing JIT's emission is input-dependent. Copy-and-patch's public fixed stencil set, by contrast, narrows the candidate gadget patterns, their semantics, and their origins before the target runs. This reduction, however, is confined to *which* gadgets exist; it does not lower the runtime entropy of absolute addresses, layout, patched bytes, or the probabilistic availability of the bottleneck gadgets (ASLR included), so the attacker must still use the assumed disclosure to locate and inspect emitted buffers, match runtime-dependent patterns, and resolve addresses. (2) *Chosen data versus engine/allocator bytes*: the evaluated stencils do not copy an attacker-chosen Python constant value verbatim into executable memory; the reused bytes come from structural opcodes or allocator-chosen pointers, not from an attacker-chosen constant value. (An index-writing uop would instead embed a source-influenced `co_consts` index inline, a separate and constrained chosen-immediate vector, but the optimizer never emits it; see Section 6.2.) These observations are specific to the evaluated CPython design and do not infer properties of unevaluated data-centric attacks or other JIT architectures.

6.5. Evolution of the python attack surface

Historical context. Interpreter-only CPython does not generate JIT code, although the interpreter, extensions, and shared libraries already provide native executable code. Other Python implementations have long included JIT compilers, so CPython's copy-and-patch JIT is not Python's first JIT attack surface.

Copy-and-patch as a distinct case. CPython 3.13 introduced an experimental, opt-in copy-and-patch JIT in the reference implementation. Its fixed stencil bytes, known gadget semantics, and categories of runtime patch sites can be cataloged offline before a target execution. This does not make the emitted region static or fully predictable: selected stencils, runtime-patched bytes, gadget presence, layout, and absolute addresses remain execution-dependent and must be observed through the assumed disclosure primitive. StencilLeak characterizes this architecture-specific source of prior knowledge and demonstrates harness-assisted chain execution in a conditional pipeline that assumes an independent memory-safety vulnerability. We make no empirical claim that copy-and-patch is more predictable or more vulnerable than other JIT designs.

6.6. Limitations and future work

Our study focuses exclusively on the CPython implementation of the copy-and-patch JIT running on the x86_64 architecture. While template determinism represents a fundamental structural vulnerability, its practical exploitability heavily depends on the underlying instruction set architecture (ISA). For instance, fixed-length instruction encodings such as AArch64 (the A64 execution state of ARMv8) are inherently immune to unaligned instruction decoding (Roemer et al., 2012). In such fixed-length environments, attackers would be strictly limited to the aligned gadgets explicitly compiled into the stencils, significantly raising the exploitation bar. By contrast, the variable-length AArch32/Thumb-2 execution state of ARMv8 (2 or 4 bytes per instruction) is not immune to unaligned decoding, so it would not impose this restriction. Future work should investigate how template determinism manifests on RISC architectures, and whether semantic gadget chaining remains viable without the expanded gadget pool afforded by variable-length execution.

6.7. Vulnerability disclosure

We notified the Python Security Response Team (PSRT) in January 2026 of our findings and received approval of our coordinated disclosure plan. To minimize risk to end users, technical details and proof-of-concept exploit code have been withheld pending remediation; the PSRT will determine the timing and scope of public release. We will continue to coordinate with PSRT after peer review to ensure patches are tested and deployed before full disclosure. This coordinated approach follows responsible-disclosure best practices and aims to balance transparency with the need to prevent premature exploitation.

7. Conclusion

Our findings show that enabling CPython's experimental copy-and-patch JIT adds a partially offline-cataloguable code-reuse surface alongside the native code already present in the interpreter, extensions, and shared libraries. StencilLeak combines offline knowledge of fixed stencil bytes and patch-site categories with runtime disclosure of the selected stencils, final patched bytes, layout, and addresses. Unlike data-centric JIT spraying, source structure influences which stencils are emitted, but the reused bytes come from fixed opcodes and engine- or allocator-controlled pointer immediates rather than attacker-chosen values. The resulting surface is only partially predictable: structural gadgets are deterministic, whereas the `pop rax; ret` and `pop rdx; ret` bottlenecks arise from ASLR-dependent runtime patching and are therefore probabilistic.

In a 500-process benchmark using a fixed, GC-disabled 3000-function workload and a two-hour per-process search budget, 251 runs (50.2%) located all five required gadget patterns. Separately, after an instrumentation harness supplied repeatable disclosure, writable chain storage, a controlled stack, and the initial control transfer, the PoC constructed and executed a native `execve` chain from JIT-resident gadgets. These experiments establish the feasibility of the JIT-resident code-reuse stage after an assumed control-flow hijack, not an end-to-end memory-corruption exploit. We further show, in the Discussion, that the assumed disclosure, write, and control-transfer primitives are concretely realizable by a public, already-patched CPython use-after-free (CVE-2026-6100), which does not by itself supply the controlled stack a full launch also needs. Template determinism alone neither defeats ASLR nor demonstrates a strict W^X bypass: runtime disclosure remains necessary, and the synthetic launcher—not the RX gadget chain—uses RWX memory. Userspace shadow stacks, forward-edge CFI on the launch transfer, and syscall filtering therefore remain direct controls against the demonstrated chain; evaluating low-overhead changes to CPython's JIT remains future work.

CRediT authorship contribution statement

Seehwan Yoo: Writing – original draft, Supervision, Project administration, Investigation, Funding acquisition, Formal analysis, Conceptualization. **Youngpil Kim:** Writing – review & editing, Validation, Software, Methodology, Formal analysis, Conceptualization. **Chuck Yoo:** Writing – review & editing, Supervision, Resources, Project administration, Data curation.

Ethical considerations

We followed a coordinated-vulnerability-disclosure process: (1) privately reported our findings to the vendor security response team in January 2026; (2) provided minimal proof-of-concept details sufficient for reproduction by maintainers; and (3) withheld full exploit code and offsets until patches ship. No production systems were probed; all experiments ran on isolated test hosts we controlled. We used only stock CPython builds and public dependencies, avoided third-party data, and performed no user tracking or data collection.

Dual-use risk is inherent in exploitation research. We mitigate it by limiting released artifacts to evaluation harnesses and synthetic inputs that demonstrate the issue without enabling turnkey exploitation, and by committing to release timelines aligned with patch availability. The paper omits sensitive addresses and heap/stack layouts; where exact offsets are needed for scientific claims, we aggregate statistics instead of publishing per-run traces. We also recommend deployable mitigations (Section 6) to reduce immediate risk to operators.

No human or animal subjects were involved. Resource usage was modest (single-tenant lab machines), and environmental impact was negligible. We verified that our disclosure plan complies with the conference ethics guidelines and preserves anonymity for double-blind review.

Open science

To facilitate future research and ensure reproducibility, we provide our research artifacts, including the exploit harness, analysis scripts, and evaluation data, via the following public repository: https://doi.org/10.5281/zenodo.23569119

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work, the author(s) used ChatGPT for language editing. The author(s) reviewed and edited the output as needed and take full responsibility for the content of the published article.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No. RS-2026-25518394). This work was supported by the Ministry of Science and ICT (MSIT), Korea, through Information Technology Research Center (ITRC) Program (IITP-2026-RS-2023-00259061); Institute for Information and Communications Technology Planning and Evaluation (IITP). This research was supported by Basic Science Research Program through the National Research Foundation of Korea (NRF) funded by the Ministry of Education (RS-2021-NR060143).

Data availability

Data will be made available on request.

References

Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., Devin, M., Ghemawat, S., Irving, G., Isard, M., et al., 2016. TensorFlow: A system for large-scale machine learning. In: 12th USENIX Symposium on Operating Systems Design and Implementation. OSDI, pp. 265–283.

Abadi, M., Budiu, M., Erlingsson, U., Ligatti, J., 2005. Control-flow integrity: Principles, implementations, and applications. In: ACM Conference on Computer and Communications Security. CCS.

Apple Inc., 2024. mmap(2) — Map files or devices into memory (MAP_JIT). https://github.com/apple/darwin-xnu/blob/main/bsd/man/man2/mmap.2. (Accessed 31 October 2025).

Athanasakis, M., Athanasopoulos, E., Polychronakis, M., Portokalidis, G., Ioannidis, S., 2015. The devil is in the constants: Bypassing defenses in browser jit engines. In: Network and Distributed System Security Symposium. NDSS.

Bigelow, D., Hobson, T., Rudd, R., Streilein, W., Okhravi, H., 2015. Timely rerandomization for mitigating memory disclosures. In: ACM Conference on Computer and Communications Security. CCS, pp. 268–279. http://dx.doi.org/10.1145/2810103.2813691.

Blazakis, D., 2010. Interpreter exploitation: Pointer inference and JIT spraying. In: Black Hat DC.

Bolz, C.F., Rigo, A., 2009. Tracing the meta-level: PyPy's tracing JIT compiler. ACM SIGPLAN Not. 44 (12), 18–25.

Bucher, B., Shannon, M., 2023. PEP 744 – JIT compilation for CPython. https://peps.python.org/pep-0744/.

Chen, P., Fang, Y., Mao, B., Xie, L., 2011. JITDefender: A defense against JIT spraying attacks. In: IFIP Int'L Information Security Conference.

Frassetto, T., Gens, D., Liebchen, C., Sadeghi, A.-R., 2017. JITGuard: Hardening just-in-time compilers with SGX. In: ACM Conference on Computer and Communications Security. CCS.

Gawlik, R., Holz, T., 2018. SoK: Make JIT-Spray Great Again. In: USENIX Workshop on Offensive Technologies. WOOT.

Groß, S., Koch, S., Bernhard, L., Holz, T., Johns, M., 2023. FUZZILLI: Fuzzing for JavaScript JIT compiler vulnerabilities. In: Network and Distributed System Security Symposium. NDSS.

He, Y., et al., 2023. Flow hijacking in eBPF: Exploitation and mitigation across both interpreter and JIT execution. In: Proceedings of the Network and Distributed System Security Symposium. NDSS.

Holler, C., Herzig, K., Zeller, A., 2012. Fuzzing with code fragments. In: USENIX Security Symposium.

Homescu, A., Brunthaler, S., Larsen, P., Franz, M., 2013. Librando: Transparent code randomization for just-in-time compilers. In: Proceedings of the 2013 ACM SIGSAC Conference on Computer & Communications Security. CCS, ACM, pp. 993–1004. http://dx.doi.org/10.1145/2508859.2516675.

Jauernig, M., Neugschwandtner, M., Platzer, C., Comparetti, P.M., 2014. Lobotomy: An architecture for JIT spraying mitigation. In: Intl. Conf. on Availability, Reliability and Security. ARES.

Lehmann, D., Kinder, J., Pradel, M., 2020. Everything old is new again: Binary security of WebAssembly. In: 29th USENIX Security Symposium. USENIX Security 20, pp. 217–234.

Lian, W., Shacham, H., Savage, S., 2015. Too LeJIT to quit: Extending JIT spraying to ARM. In: Network and Distributed System Security Symposium. NDSS.

Liljestrand, H., Nyman, T., Wang, K., Pérez, C.C., Ekberg, J., Asokan, N., 2019. PAC it up: Towards pointer integrity using ARM pointer authentication. In: 28th USENIX Security Symposium. USENIX Security 2019.

Maisuradze, G., Backes, M., Rossow, C., 2016. What cannot be read, cannot be leveraged? Revisiting assumptions of JIT-ROP defenses. In: USENIX Security Symposium.

MesaPy Contributors, 2019. MesaPy: Memory-Safe PyPy variant for enclaves. https://github.com/mesalock-linux/mesapy. (Accessed 31 October 2025).

Niu, B., Tan, G., 2014. Rockjit: Securing just-in-time compilation using modular control-flow integrity. In: ACM Conference on Computer and Communications Security. CCS.

Numba Developers, 2024. Numba: A just-in-time compiler for Python. https://numba.pydata.org/. (Accessed 31 October 2025).

Onarlioglu, K., Bilge, L., Lanzi, A., Balzarotti, D., Kirda, E., 2010. G-Free: Defeating return-oriented programming through gadget-less binaries. In: Proceedings of the 26th Annual Computer Security Applications Conference. ACSAC, ACM, pp. 49–58. http://dx.doi.org/10.1145/1920261.1920269.

One, A., 1996. Smashing the stack for fun and profit. Phrack 7, URL https://phrack.org/issues/49/14.

Pappas, V., Polychronakis, M., Keromytis, A.D., 2012. Smashing the gadgets: Hindering return-oriented programming using in-place code randomization. In: Proceedings of the 2012 IEEE Symposium on Security and Privacy (S&P). IEEE Computer Society, pp. 601–615. http://dx.doi.org/10.1109/SP.2012.41.

Park, T., Dhondt, K., Gens, D., Na, Y., Volckaert, S., Franz, M., 2020. NoJITsu: Locking down JavaScript engines. In: Network and Distributed System Security Symposium. NDSS.

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., et al., 2019. PyTorch: An imperative style, high-performance deep learning library. In: Advances in Neural Information Processing Systems, Vol. 32. NeurIPS.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., et al., 2011. Scikit-learn: Machine learning in Python. J. Mach. Learn. Res. 12, 2825–2830.

Pyston Team, 2021. Pyston: A faster implementation of Python. https://github.com/pyston/pyston. (Accessed 31 October 2025).

Python Core Developers, 2023. Faster CPython project: JIT experimental performance results. https://github.com/faster-cpython/ideas/issues/280.

Python Software Foundation, 2024. What's new in Python 3.13. https://docs.python.org/3.13/whatsnew/3.13.html.

PyTorch Team, 2023. TorchDynamo: Python-first graph capture. https://docs.pytorch.org/docs/2.14/user_guide/torch_compiler/torch.compiler_dynamo_overview.html. (Accessed 26 October 2026).

Roemer, R., Buchanan, E., Shacham, H., Savage, S., 2012. Return-oriented programming: Systems, languages, and applications. In: ACM Transactions on Information and System Security, Vol. 15, No. 1. TISSEC, ACM New York, NY, USA, pp. 1–34.

Shanbhogue, V., Gupta, D., Sahita, R., 2019. Security analysis of processor instruction set architecture for enforcing control-flow integrity. In: Proceedings of the 8th International Workshop on Hardware and Architectural Support for Security and Privacy. HASP, ACM, http://dx.doi.org/10.1145/3337167.3337175.

Shannon, M., et al., 2021. PEP 659 – specializing adaptive interpreter. In: Python Enhancement Proposals. https://peps.python.org/pep-0659/.

Snow, K.Z., Monrose, F., Davi, L., Dmitrienko, A., Liebchen, C., Sadeghi, A., 2013. Just-in-time code reuse: On the effectiveness of fine-grained address space layout randomization. In: IEEE Symposium on Security and Privacy. S&P.

Stinner, V., 2013. The pysandbox project is broken. https://mail.python.org/pipermail/python-dev/2013-November/130132.html. Message to the Python-Dev mailing list, 12 November 2013.

Wang, H., Bauman, E., Karande, V., Lin, Z., Cheng, Y., Zhang, Y., 2019. Running language interpreters inside SGX: A lightweight, legacy-compatible script code hardening approach. In: Proc. of the 2019 ACM Asia Conference on Computer and Communications Security. AsiaCCS, pp. 114–121. http://dx.doi.org/10.1145/3321705.3329848.

Wang, J., Zhang, Z., Liu, S., Du, X., Chen, J., 2023. FuzzJIT: Oracle-enhanced fuzzing for JavaScript engine JIT compiler. In: 32nd USENIX Security Symposium. USENIX Security 2023.

Xu, H., Kjolstad, F., 2021. Copy-and-patch compilation: A fast method for custom code generation. In: Proceedings of the ACM on Programming Languages, Vol. 5. OOPSLA, ACM, pp. 1–30.

Yee, B., Sehr, D., Dardyk, G., Chen, J.B., Muth, R., Ormandy, T., Okasaka, S., Narula, N., Fullagar, N., 2009. Native client: A sandbox for portable, untrusted x86 native code. In: IEEE Symposium on Security and Privacy. S&P, pp. 79–93.

Zhang, J., Chen, Y., Gong, X., Zhang, J., Wang, W., Yew, P.-C., 2024. JiuJITsu: Removing gadgets with safe register allocation for JIT code generation. ACM Trans. Archit. Code Optim. (TACO) 21 (1).